



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**

PROJET DE FIN D'ANNÉE

IoT Network-Flow

Intrusion Detection

RÉALISÉ PAR :

Zakaria El Khaldi

TUTEUR(S) :

Encadrant professionnel :

Non applicable – projet académique

Encadrant pédagogique :

Dr. Anass Mancourbillah

3^e année en Ingénierie Informatique et Réseaux

Année universitaire **2026–2027**

Remerciements

Nous adressons nos remerciements à l'équipe pédagogique de l'EMSI pour le cadre de formation qui a rendu possible la réalisation de ce projet d'été. Les enseignements consacrés au génie logiciel, aux réseaux, à la sécurité informatique et à l'apprentissage automatique ont fourni les bases nécessaires à sa conception.

Nous remercions également les auteurs des travaux scientifiques et des outils libres cités dans ce rapport. Leurs publications, documentations et logiciels ont permis de construire une démarche reproductible et de confronter les choix du prototype à des références vérifiables.

Enfin, nous exprimons notre reconnaissance à toutes les personnes ayant relu le travail ou formulé des observations utiles à son amélioration.

Résumé

Ce projet présente un prototype reproductible de détection d'intrusions fondé sur des observations de flux réseau IoT. Le jeu de données RT-IoT2022 est contrôlé, dédoublé et transformé selon un schéma versionné. Une cascade supervisée associe un détecteur binaire à un classifieur multiclasse de familles d'attaque. Les candidats sont comparés sur des partitions distinctes d'entraînement, de validation et de test, avec calibration et conservation des artefacts promus.

Le prototype comprend une API FastAPI, une ingestion durable et idempotente, une persistance transactionnelle, une diffusion temps réel après validation et une interface React destinée à la supervision et à l'investigation. Une voie Suricata distincte conserve les alertes issues de signatures sans les confondre avec les prédictions du modèle. Le système restitue aussi les versions des artefacts, les preuves d'évaluation et des indicateurs de santé du modèle.

Sur le partage de test interne RT-IoT2022, la cascade atteint un macro-F1 de 0,9520 pour la graine 42 et une moyenne de 0,9686 sur trois graines. Ce résultat ne constitue toutefois pas une validation sur un réseau IoT réel. L'absence de corpus NFStream natif étiqueté et d'évaluation externe demeure la principale limite du travail.

Mots-clés : Internet des objets, détection d'intrusions, flux réseau, apprentissage automatique, RT-IoT2022, reproductibilité.

Abstract

This project presents a reproducible intrusion-detection prototype based on IoT network-flow observations. The RT-IoT2022 dataset is verified, deduplicated, and transformed under a versioned feature schema. A supervised cascade combines a binary detector with a multiclass attack-family classifier. Candidate models are compared using separate training, validation, and test partitions, with probability calibration and controlled promotion of the resulting artifacts.

The prototype includes a FastAPI service, durable and idempotent ingestion, transactional persistence, post-commit real-time publication, and a React interface for monitoring and investigation. A separate Suricata path preserves signature-based alerts without presenting them as model predictions. The system also exposes artifact versions, evaluation evidence, and model-health indicators.

On the declared internal RT-IoT2022 test split, the complete cascade reaches a macro-F1 score of 0.9520 for seed 42 and a three-seed mean of 0.9686. This result does not demonstrate effectiveness on a real IoT network. The absence of a labelled native NFStream corpus and of external evaluation remains the principal limitation of the work.

Keywords: Internet of Things, intrusion detection, network flows, machine learning, RT-IoT2022, reproducibility.

Table des matières

Remerciements

Résumé

Abstract

Introduction générale	1
1 Cadre scientifique et problématique de recherche	2
1.1 Introduction	2
1.2 Contexte scientifique	2
1.2.1 Détection d'intrusions dans les environnements IoT	2
1.2.2 Jeu de données RT-IoT2022	3
1.3 Analyse des approches existantes	4
1.3.1 Détection par signatures et détection statistique	4
1.3.2 Classification supervisée des flux	5
1.3.3 Limites des validations usuelles	5
1.4 Problématique et questions de recherche	6
1.5 Solution proposée et périmètre	6
1.6 Démarche de recherche	7
1.6.1 Mode de développement	7
1.6.2 Phases de recherche	7
1.7 Conclusion	9
2 Spécification des besoins	10
2.1 Introduction	10
2.2 Besoins fonctionnels	10

2.3	Besoins non fonctionnels	11
2.4	Présentation des cas d'utilisation	11
2.5	Présentation des acteurs	11
2.6	Diagramme global des cas d'utilisation	12
2.7	Description des cas d'utilisation	13
2.7.1	UC-02 — Submit network flow observations	13
2.7.2	UC-03 — Submit Suricata sensor events	13
2.7.3	UC-04 — Control dataset replay	14
2.7.4	UC-05 — Investigate alerts and record disposition	14
2.8	Conclusion	15
3	Conception du système	16
3.1	Introduction	16
3.2	Modélisation statique	16
3.2.1	Modèle du domaine	16
3.2.2	Modèle relationnel	20
3.2.3	Dictionnaire de données	20
3.3	Architecture de l'application	22
3.4	Modélisation dynamique	24
3.4.1	Activité de l'opérateur	24
3.4.2	Communication pendant l'investigation	24
3.4.3	Ingestion durable des observations	24
3.4.4	Prédiction synchrone	28
3.4.5	Investigation des alertes	29
3.4.6	Surveillance de la santé des modèles	30
3.4.7	Scénarios couverts sans diagramme supplémentaire	30
3.5	Conclusion	30
4	Réalisation du système	31
4.1	Introduction	31

4.2	Environnement de développement	31
4.2.1	Environnement matériel	31
4.2.2	Environnement logiciel	31
4.3	Organisation de la réalisation	32
4.4	Principaux choix d'implémentation	32
4.4.1	Du modèle entraîné à la prédiction servie	33
4.4.2	Ingestion durable et traitement des erreurs	33
4.4.3	Interface, contrôle d'accès et provenance	33
4.4.4	Exécution reproductible	34
4.5	Principales interfaces graphiques	35
4.5.1	Supervision opérationnelle	35
4.5.2	Investigation des alertes	36
4.5.3	Analyse des modèles	37
4.5.4	Laboratoire d'observations	38
4.6	Validation technique	38
4.7	Conclusion	39
5	Résultats expérimentaux	40
5.1	Objectif et source de preuve	40
5.2	Qualité, nettoyage et déséquilibre	40
5.3	Protocole expérimental	40
5.4	Comparaison des classifieurs	41
5.5	Expérience sur le déséquilibre	42
5.6	Erreurs des champions et de la cascade	42
5.7	Calibration, seuil et coût opérationnel	43
5.8	Menaces à la validité	43
5.9	Conclusion	43
	Conclusion générale	44
	Bibliographie et webographie	45

Liste des figures

1.1	Architecture de référence d'un IDPS réseau passif	3
1.2	Exemple d'architecture de capture IoT	5
1.3	Comparaison des modèles dans le carnet de recherche exécuté	8
1.4	Phases indicatives de réalisation et chevauchement des travaux	8
2.1	Cas d'utilisation du système de détection	12
3.1	Diagramme de classes du domaine principal	17
3.2	Classes associées à l'ingestion durable	18
3.3	Classes associées au suivi de l'état des modèles	19
3.4	Services et opérations principales	20
3.5	Architecture logicielle par composants	22
3.6	Déploiement du mode de démonstration Docker	23
3.7	Activité principale de l'opérateur	25
3.8	Communication entre objets lors de l'investigation d'une alerte	26
3.9	Séquence d'ingestion durable et de diffusion des résultats	27
3.10	Machine à états d'un travail d'ingestion	27
3.11	Séquence de prédiction synchrone	28
3.12	Séquence d'investigation et de qualification d'une alerte	29
3.13	Séquence de consultation de la santé d'un modèle	30
4.1	Espace principal de supervision du prototype	35
4.2	Espace d'investigation des alertes	36
4.3	Espace d'analyse et de traçabilité des modèles	37
4.4	Laboratoire de validation et de soumission d'observations	38
5.1	Comparaison des candidats dans le notebook d'analyse	41

Liste des tableaux

1.1	Résultats de l’audit local du fichier RT-IoT2022 contrôlé	4
2.1	Besoins fonctionnels du prototype	10
2.2	Besoins non fonctionnels et critères observables	11
2.3	Description du cas UC-02 — Submit network flow observations	13
2.4	Description du cas UC-03 — Submit Suricata sensor events	13
2.5	Description du cas UC-04 — Control dataset replay	14
2.6	Description du cas UC-05 — Investigate alerts and record disposition	14
3.1	Dictionnaire synthétique des données persistées	20
4.1	Environnement logiciel et rôle des principaux outils	31
5.1	Comparaison du détecteur binaire sur les trois graines	41
5.2	Comparaison du classifieur des familles d’attaque	41
5.3	Comparaison contrôlée des traitements du déséquilibre	42
5.4	Classes les plus fragiles dans la cascade, graine 42	42

Introduction générale

Les objets connectés produisent des communications nombreuses et hétérogènes dont l'analyse manuelle devient rapidement impraticable. La détection d'intrusions fondée sur les flux réseau offre une observation moins intrusive que l'inspection du contenu, mais elle impose trois exigences: un schéma de données stable, une évaluation scientifique reproductible et une chaîne de traitement capable de conserver les preuves utiles à l'investigation.

Ce projet d'étude étudie la question suivante: comment construire un prototype reproductible qui classe des observations de flux IoT, signale les comportements suspects et présente à l'opérateur les informations nécessaires à leur examen, sans confondre un résultat obtenu sur RT-IoT2022 avec une validation en environnement de production? Le travail couvre la préparation contrôlée du jeu de données, l'évaluation d'une cascade supervisée, l'ingestion durable, la persistance des résultats et une interface d'investigation. La collecte NFStream en direct et l'efficacité sur un réseau cible restent hors du périmètre validé.

Le rapport est organisé en quatre chapitres. Le premier établit le cadre scientifique, la problématique et la démarche de recherche. Le deuxième formalise les besoins et les cas d'utilisation. Le troisième présente la conception dynamique, statique et architecturale. Le quatrième décrit l'environnement de réalisation, les principaux choix d'implémentation et les interfaces essentielles du prototype.

CHAPITRE 1

Cadre scientifique et problématique de recherche

1.1 Introduction

Ce projet d'étude a été mené dans un cadre académique et autonome. Ce chapitre en établit le cadre scientifique à travers une étude du contexte, des travaux existants et des limites de validité qui structurent la recherche.

L'objectif est de définir précisément le problème traité par le prototype *IoT Network-Flow Intrusion Detection*. Nous présentons d'abord le cadre de la détection d'intrusions dans l'Internet des objets, puis le jeu de données retenu et les principales approches d'apprentissage automatique. Cette analyse permet ensuite de formuler la problématique, de délimiter la contribution du projet et d'exposer la méthodologie suivie.

1.2 Contexte scientifique

1.2.1 Détection d'intrusions dans les environnements IoT

Un système de détection et de prévention d'intrusions observe des événements sur un hôte ou un réseau afin d'identifier des activités susceptibles de compromettre la confidentialité, l'intégrité ou la disponibilité des ressources (Scarfone and Mell, 2007). Dans un environnement IoT, cette fonction est rendue plus difficile par l'hétérogénéité des appareils, des protocoles et des capacités matérielles. La littérature distingue notamment les méthodes selon la source des données, la stratégie de détection et le positionnement du détecteur dans l'architecture IoT (Zarpelão et al., 2017).

Le présent projet se limite aux observations agrégées par flux réseau. Une observation décrit les deux sens d'une communication au moyen de compteurs, de durées, de statistiques de paquets et d'attributs protocolaires. Cette représentation est plus compacte qu'une analyse du contenu de chaque paquet, mais sa valeur dépend directement de la définition et de la reproductibilité de l'extracteur. Des outils tels que NFStream permettent de construire des mesures de flux programmables; ils ne garantissent toutefois pas, à eux seuls, l'équivalence avec les variables d'un jeu de données produit par un autre extracteur (Aouini and Pekar, 2022).

La figure 1.1 fournit un repère architectural pour l'observation passive du trafic. Elle illustre

plusieurs points de copie, des capteurs IDPS et un réseau de gestion séparé. Elle n'est pas présentée comme l'architecture du prototype: elle sert uniquement à situer le principe de collecte hors bande décrit par le NIST.

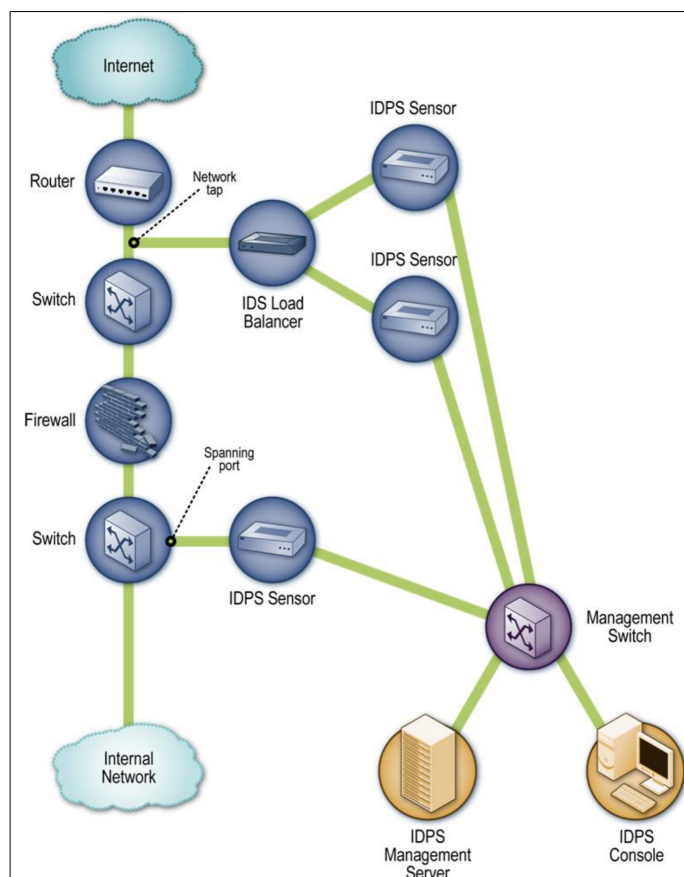


Figure 1.1 – Architecture de référence d'un IDPS réseau passif. Source: figure 4-3 de Scarfone and Mell (2007).

1.2.2 Jeu de données RT-IoT2022

RT-IoT2022 a été choisi comme support expérimental. La notice UCI le présente comme un jeu de données tabulaire multivarié consacré au trafic IoT et publie 123 117 observations comportant 83 variables (Sharmila and Nagapadma, 2023a). Le travail associé étudie également un autoencodeur quantifié destiné à la détection d'anomalies sur des dispositifs aux ressources contraintes (Sharmila and Nagapadma, 2023b).

Dans ce projet, les faits expérimentaux proviennent du fichier CSV officiel contrôlé par somme SHA-256, et non d'une reconstruction de la description en ligne. L'audit local confirme 123 117 lignes, 83 variables de modèle et douze étiquettes observées: trois classes de trafic normal et neuf familles d'attaque. Il relève également 5 202 vecteurs de caractéristiques dupliqués et une forte concentration de la classe DOS_SYN_Hping. Ces propriétés imposent une déduplication avant la séparation des données et une lecture prudente des scores agrégés.

Tableau 1.1 – Résultats de l’audit local du fichier RT-IoT2022 contrôlé

Propriété vérifiée	Valeur observée
Observations sources	123 117
Variables utilisées par les modèles	83
Étiquettes observées	12
Observations normales	12 507
Observations d’attaque	110 610
Vecteurs de caractéristiques dupliqués	5 202

La provenance de l’extraction constitue une limite importante. Les sources publiques ne donnent pas une description suffisamment cohérente des outils, unités, temporisations et règles directionnelles pour affirmer qu’un nouveau flux extrait localement est numériquement compatible avec les 83 variables publiées. En outre, le tableau ne contient pas d’identifiant fiable de session, d’appareil ou de temps permettant une séparation chronologique ou groupée.

Une validation externe demanderait donc une chaîne de capture documentée et reproductible. La figure 1.2 montre un exemple officiel d’environnement de capture pour appareils IoT filaires et sans fil. Il s’agit d’une architecture méthodologique de référence, et non de l’environnement ayant produit RT-IoT2022 ni d’un composant actuellement déployé par ce projet.

1.3 Analyse des approches existantes

1.3.1 Détection par signatures et détection statistique

La détection par signatures recherche des motifs connus. Elle offre une justification directe lorsqu’une règle correspond, mais reste dépendante de la couverture et de la mise à jour de cette base de règles. La détection statistique ou par apprentissage cherche au contraire des régularités dans les données. Elle peut reconnaître des configurations plus complexes, au prix d’une dépendance forte envers les données d’entraînement et le contexte de collecte. Ces familles sont complémentaires; le laboratoire Suricata du projet est donc considéré comme une source de preuves par signatures distincte des prédictions du modèle RT-IoT2022.

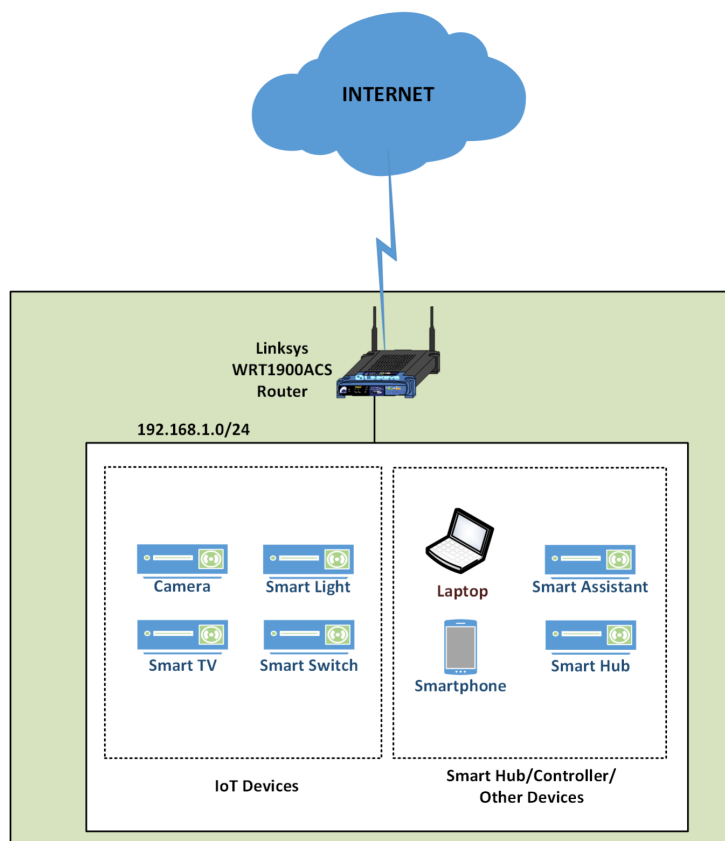


Figure 1.2 – Exemple d’architecture de capture IoT. Source: figure 4 de Watrobski et al. (2025).

1.3.2 Classification supervisée des flux

Le banc d’essai du projet compare quatre familles disponibles dans Scikit-learn (Pedregosa et al., 2011): régression logistique, arbre de décision, forêt aléatoire et gradient boosting par histogrammes. Les forêts aléatoires combinent plusieurs arbres construits avec de l’aléa afin de réduire la variance (Breiman, 2001); le gradient boosting construit progressivement un modèle additif optimisant une fonction de perte (Friedman, 2001).

L’architecture retenue est une cascade à deux étages. Le premier classifieur sépare les flux normaux des attaques. Le second n’est sollicité que pour les flux classés comme attaques et attribue alors une famille. Cette séparation reflète le chemin d’inférence réel: une attaque manquée par le premier étage ne peut recevoir aucune famille au second. Par conséquent, l’évaluation doit publier à la fois les résultats de chaque étage et ceux de la cascade complète.

1.3.3 Limites des validations usuelles

Une erreur de protocole peut produire un score optimiste sans améliorer le système réel. La fuite de données apparaît lorsqu’une information indisponible au moment de la prédiction influence l’entraînement ou la sélection du modèle (Kaufman et al., 2012). Dans le cas présent, le prétraitement doit donc être ajusté uniquement sur l’ensemble d’entraînement, et les doublons

exacts doivent être traités avant la partition.

La généralisation entre environnements est une seconde menace. Une étude inter-jeux de données rapporte que des classifieurs presque parfaits dans leur jeu d'origine peuvent perdre fortement en performance lorsqu'ils sont appliqués à une autre collecte (Cantone et al., 2024). Ce résultat ne mesure pas directement RT-IoT2022, mais il interdit d'assimiler une validation aléatoire sur un seul fichier à une validation sur un réseau de déploiement.

Enfin, l'exactitude seule est insuffisante lorsque les classes sont déséquilibrées. Le projet privilégie donc la macro-F1, les matrices de confusion, le support et le rappel par classe, ainsi que le taux de faux positifs. Ces mesures répondent à des aspects différents d'une tâche de classification (Sokolova and Lapalme, 2009). Les scores probabilistes sont calibrés par validation croisée limitée aux données d'entraînement, puisque la qualité du classement et celle des probabilités constituent deux propriétés distinctes (Niculescu-Mizil and Caruana, 2005).

1.4 Problématique et questions de recherche

La problématique retenue est la suivante:

Comment construire un prototype reproductible capable de classer des flux RT-IoT2022 en deux étapes, de conserver les preuves de ses décisions et de signaler honnêtement les limites qui séparent une bonne performance interne d'une validation sur un réseau IoT réel?

Cette problématique se décompose en quatre questions:

1. comment garantir la provenance, le schéma et l'ordre des variables utilisées pendant l'entraînement et l'inférence;
2. quelle combinaison de détecteur binaire et de classifieur de familles offre le meilleur compromis mesuré sur le protocole déclaré;
3. quelles erreurs sont introduites par chacun des deux étages et par la cascade complète;
4. quelles preuves supplémentaires seraient nécessaires avant toute affirmation de détection sur un réseau externe ou en temps réel.

1.5 Solution proposée et périmètre

La solution proposée est un prototype de recherche reproductible composé de quatre chaînes cohérentes. La première prépare et contrôle le jeu de données, supprime les doublons selon une politique explicite et produit un schéma versionné. La deuxième entraîne plusieurs candidats,

sélectionne les modèles sur la validation et réserve le test à l'estimation finale. La troisième sert la cascade au moyen d'une API, persiste les observations, prédictions et alertes, puis transmet les événements après validation. La quatrième fournit une interface d'investigation, des explications locales et des indicateurs de santé du modèle.

Le périmètre doit rester explicite. Le système classe des enregistrements de flux validés et rejoue des observations enregistrées. Il ne prouve pas que le modèle RT-IoT2022 est compatible avec un flux capturé par NFStream, ni qu'il se généralise à un autre réseau. Le laboratoire Suricata analyse de vrais paquets dans un environnement isolé, mais ses alertes de signatures ne sont pas des prédictions du modèle. Ces frontières empêchent de transformer une preuve d'ingénierie en affirmation scientifique non mesurée.

L'interface qui restitue ces limites et les preuves d'évaluation est présentée avec les autres vues principales au chapitre 4.

1.6 Démarche de recherche

1.6.1 Mode de développement

Le développement suit une démarche incrémentale guidée par les preuves. Chaque incrément part d'un contrat ou d'une limite explicite, ajoute les tests correspondants, implémente le comportement puis produit un artefact vérifiable. L'historique du dépôt atteste cette progression depuis la préparation des données jusqu'à l'ingestion, à la supervision et au laboratoire isolé. Cette organisation ne prétend pas appliquer un cadre Scrum ou un calendrier d'entreprise qui n'ont pas existé.

1.6.2 Phases de recherche

La conduite du projet est organisée en phases traçables:

1. audit des sources scientifiques, du fichier RT-IoT2022 et de son schéma;
2. préparation reproductible des données et définition du protocole d'évaluation;
3. comparaison des candidats, calibration, analyse des erreurs et promotion atomique des artefacts;
4. conception de l'API, de la persistance et de l'ingestion durable, puis réalisation de l'interface d'investigation;
5. tests unitaires, tests d'intégration, validation des artefacts et constitution d'un paquet de preuves reproductible;

6. identification des travaux non encore validés: collecte native NFStream, séparation par session ou par temps, validation externe et mesure sur le matériel cible.

Cette organisation constitue une démarche de recherche vérifiable. Chaque phase produit un artefact contrôlable: manifestes et sommes de contrôle, rapports d'évaluation, modèles versionnés, tests automatisés, documentation d'architecture ou preuves d'exécution.

Le carnet exécuté fournit une trace lisible de la comparaison des candidats. La figure 1.3 reproduit directement sa section de comparaison, sans ressaisir ni recalculer les valeurs dans le rapport.

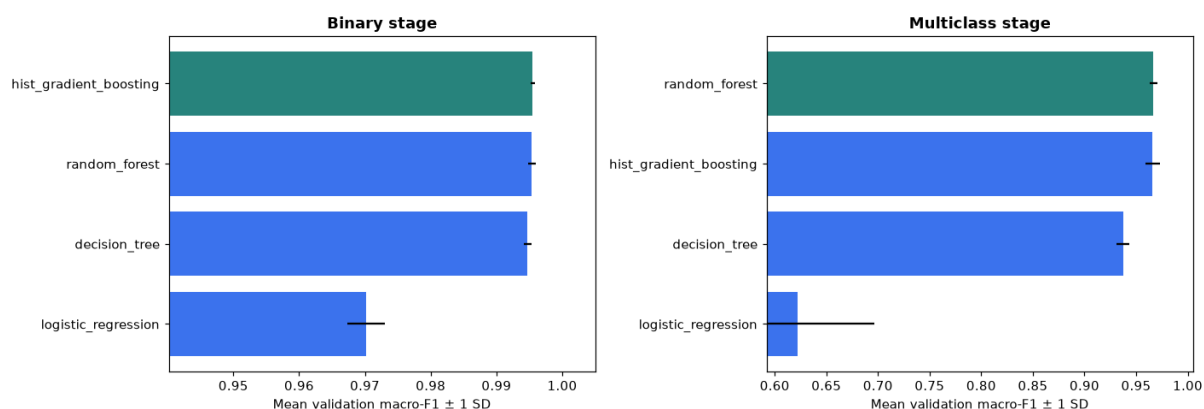


Figure 1.3 – Comparaison des modèles dans le carnet de recherche exécuté

La figure 1.4 synthétise le déroulement du projet par semaines indicatives. Le développement du tableau de bord a commencé après les premières expérimentations, puis s'est poursuivi en parallèle de la recherche et de l'intégration de l'API. Les phases se chevauchent: elles représentent les principales périodes de travail et non des durées exactes déduites des dates de commit.

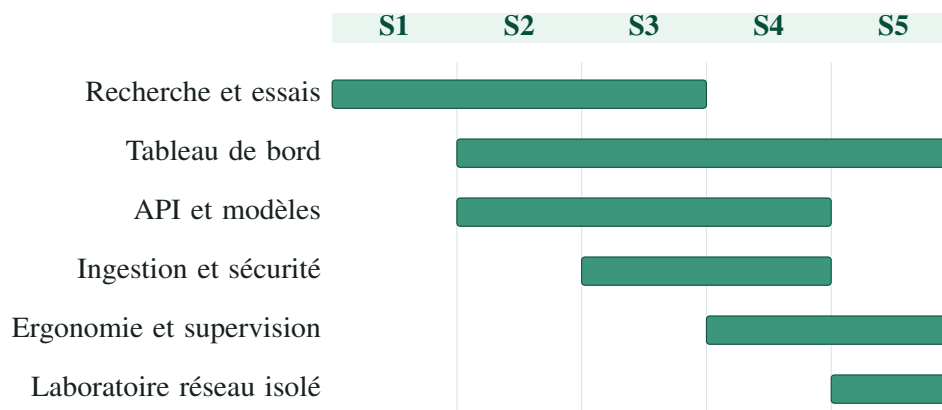


Figure 1.4 – Phases indicatives de réalisation et chevauchement des travaux

1.7 Conclusion

Ce chapitre a situé le projet dans le domaine de la détection d'intrusions sur flux IoT et a défini un cadre scientifique adapté à un travail autonome. L'analyse de RT-IoT2022 et des travaux existants justifie une cascade supervisée reproductible, tout en montrant que les scores obtenus sur un seul jeu de données ne constituent pas une validation de déploiement. La problématique, le périmètre et la démarche ainsi définis serviront de base à la spécification des besoins du système au chapitre suivant.

CHAPITRE 2

Spécification des besoins

2.1 Introduction

Ce chapitre traduit le périmètre du prototype en besoins vérifiables. Les exigences sont tirées des contrats d'API, des contrôles d'entrée, des vues opérateur et des limites déclarées dans le dépôt. Elles ne supposent ni déploiement de production ni compatibilité NFStream en direct.

2.2 Besoins fonctionnels

Tableau 2.1 – Besoins fonctionnels du prototype

ID	Besoin
BF-01	Authentifier l'opérateur et protéger les fonctions métier par un jeton à durée limitée.
BF-02	Valider puis accepter des observations de flux versionnées en mode synchrone ou par ingestion durable.
BF-03	Exécuter la cascade binaire puis, uniquement pour une attaque, la classification de famille.
BF-04	Rejouer un sous-ensemble borné du jeu de données et permettre la pause, la reprise et l'arrêt.
BF-05	Persister les observations, prédictions, alertes et transitions utiles à l'audit.
BF-06	Présenter, filtrer et détailler les alertes puis enregistrer la qualification de l'analyste.
BF-07	Publier les nouvelles prédictions et alertes aux tableaux de bord connectés après validation de la transaction.
BF-08	Recevoir les événements de signature Suricata par une voie distincte et exposer l'état du capteur.
BF-09	Restituer l'identité des artefacts, leurs mesures d'évaluation et les instantanés de santé du modèle.

2.3 Besoins non fonctionnels

Tableau 2.2 – Besoins non fonctionnels et critères observables

Qualité	Critère observable
Intégrité	Vérifier le schéma, l'ordre des caractéristiques, les sommes de contrôle des artefacts et l'identité de chaque événement.
Fiabilité	Rendre l'ingestion idempotente, conserver les transitions et appliquer des reprises bornées avant mise en lettre morte.
Sécurité	Refuser par défaut les fonctions métier non authentifiées, séparer le jeton capteur et ne pas placer de secret dans l'URL WebSocket.
Réactivité	Déporter les traitements bloquants hors de la boucle asynchrone, borner les corps HTTP, les lots, la file et les connexions temps réel.
Traçabilité	Associer chaque prédiction aux versions exactes des modèles et conserver l'identité validée de l'opérateur dans l'historique.
Interopérabilité	Utiliser un contrat JSON versionné, des API HTTP et WebSocket, et distinguer explicitement les routes non compatibles.
Utilisabilité	Fournir des états de chargement et d'erreur, une navigation clavier, une adaptation mobile et des libellés qui ne transforment pas un score en certitude.

2.4 Présentation des cas d'utilisation

Les cas d'utilisation relient les objectifs des acteurs aux fonctions du prototype. Le diagramme global donne la vue d'ensemble; les descriptions textuelles suivantes sont limitées aux parcours qui comportent des validations, des alternatives ou des effets persistants.

2.5 Présentation des acteurs

Les acteurs sont présentés brièvement, conformément au guide. L'analyste ou l'administrateur est l'acteur humain qui s'authentifie, contrôle le jeu, examine les alertes et consulte les preuves. Le système IoT externe ou l'agent capteur transmet des observations conformes au schéma ou des événements Suricata authentifiés. L'attaquant n'est pas un acteur du logiciel : son activité est une donnée observée.

2.6 Diagramme global des cas d'utilisation

La figure 2.1 relie chaque objectif à l'acteur qui le déclenche. Les quatre parcours protégés incluent l'authentification; le capteur Suricata utilise pour sa part une identité machine distincte.

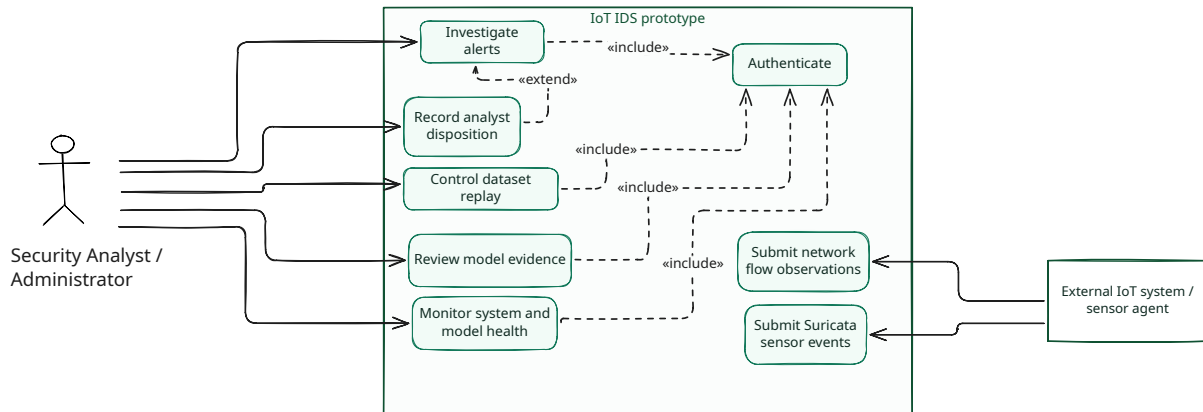


Figure 2.1 – Cas d'utilisation du système de détection

2.7 Description des cas d'utilisation

Les tableaux suivants décrivent les parcours qui possèdent des validations, des alternatives ou des effets persistants. Les chemins machine et opérateur restent séparés.

2.7.1 UC-02 — Submit network flow observations

Tableau 2.3 – Description du cas UC-02 — Submit network flow observations

Acteur	External IoT system / sensor agent utilisant le contrat d'observation autorisé.
Objectif	Soumettre d'une à mille observations canoniques à la file durable.
Préconditions	Jeton autorisé; schéma et version reconnus; identifiant d'événement fourni.
Postconditions	Chaque nouvelle observation acceptée possède un travail en file et une transition initiale; un doublon identique reste idempotent.
Scénario nominal	1. Le producteur envoie JSON ou NDJSON. 2. L'API valide la requête entière. 3. Elle contrôle identités et capacité. 4. Elle insère les travaux dans une transaction. 5. Elle retourne le reçu du lot.
Alternatives	Requête invalide: aucun travail stocké. Même identifiant avec contenu différent: conflit. File saturée: refus temporaire avec indication de reprise.

2.7.2 UC-03 — Submit Suricata sensor events

Tableau 2.4 – Description du cas UC-03 — Submit Suricata sensor events

Acteur	External IoT system / sensor agent.
Objectif	Transmettre une alerte de signature et l'état observé du capteur.
Préconditions	Jeton capteur valide; message EVE pris en charge; identifiant externe présent.
Postconditions	L'alerte et l'état du capteur sont persistés; un événement temps réel est produit après validation.

Scénario nominal	1. L'agent lit un événement Suricata. 2. Il le normalise. 3. L'API authentifie la machine. 4. Elle déduplique et persiste les preuves. 5. Elle retourne un reçu.
Alternatives	Jeton absent ou invalide: refus. Message non conforme: erreur de validation. Identifiant déjà accepté: traitement idempotent sans seconde alerte.

2.7.3 UC-04 — Control dataset replay

Tableau 2.5 – Description du cas UC-04 — Control dataset replay

Acteur	Security Analyst / Administrator.
Objectif	Exécuter un rejeu borné et en contrôler la cadence.
Préconditions	Session valide; jeu préparé disponible; aucun autre rejeu actif.
Postconditions	Les lignes traitées restent persistées; l'état final indique achèvement ou arrêt.
Scénario nominal	1. L'opérateur choisit scénario, décalage, limite et vitesse. 2. L'API accepte la configuration. 3. Le serveur filtre et traite les lignes. 4. L'interface suit la progression.
Alternatives	Configuration hors limites: refus. Rejeu actif: conflit. Pause: traitement suspendu. Reprise: cadence validée puis poursuite. Arrêt: abandon du reste sans annuler les lignes déjà traitées.

2.7.4 UC-05 — Investigate alerts and record disposition

Tableau 2.6 – Description du cas UC-05 — Investigate alerts and record disposition

Acteur	Security Analyst / Administrator.
Objectif	Examiner une alerte puis, si nécessaire, enregistrer une qualification justifiée.
Préconditions	Session valide; alerte persistée.

Postconditions

La consultation ne modifie pas l'alerte; une qualification confirmée ajoute une entrée immuable associée à l'identité authentifiée.

Scénario nominal

1. L'opérateur filtre la liste. 2. Il ouvre une alerte. 3. Il examine provenance, modèle ou signature et explication disponible. 4. Il choisit un statut. 5. Il confirme la justification.

Alternatives

Alerte absente: signalement sans modification. Explication indisponible: les autres preuves restent consultables. Justification obligatoire manquante: qualification refusée.

2.8 Conclusion

Les besoins distinguent clairement les actions humaines, les entrées machine et les garanties transversales. Cette base permet de relier chaque parcours aux composants, données et interactions décrits dans le chapitre de conception.

CHAPITRE 3

Conception du système

3.1 Introduction

Ce chapitre décrit comment les besoins sont réalisés par les composants, les interactions et les données du prototype. Les diagrammes restent en anglais pour reprendre exactement les noms techniques du code. Chaque vue isole une préoccupation afin de rester lisible sur une page A4.

3.2 Modélisation statique

3.2.1 Modèle du domaine

La vue principale de la figure 3.1 organise les concepts métier persistants: observations de flux, prédictions, alertes, jeux de données et modèles. Les associations expriment les relations structurelles; les détails propres aux traitements asynchrones sont isolés dans des vues complémentaires pour conserver une lecture exploitable sur une page A4.

Ces vues décrivent les données du domaine et omettent la visibilité des attributs. Elles n'affirment donc ni un accès public depuis l'extérieur ni une encapsulation privée imposée par Python. Le signe + est réservé aux opérations publiques des services. La protection des données repose sur l'authentification, les contrôles d'accès des routes et les transactions du backend, et non sur la notation d'un attribut dans un diagramme.

Les entités persistantes portent les données et leurs associations. Les opérations sont volontairement regroupées dans les services qui exécutent les transactions et les règles métier. La figure 3.4 présente uniquement leurs opérations publiques attestées par le code.

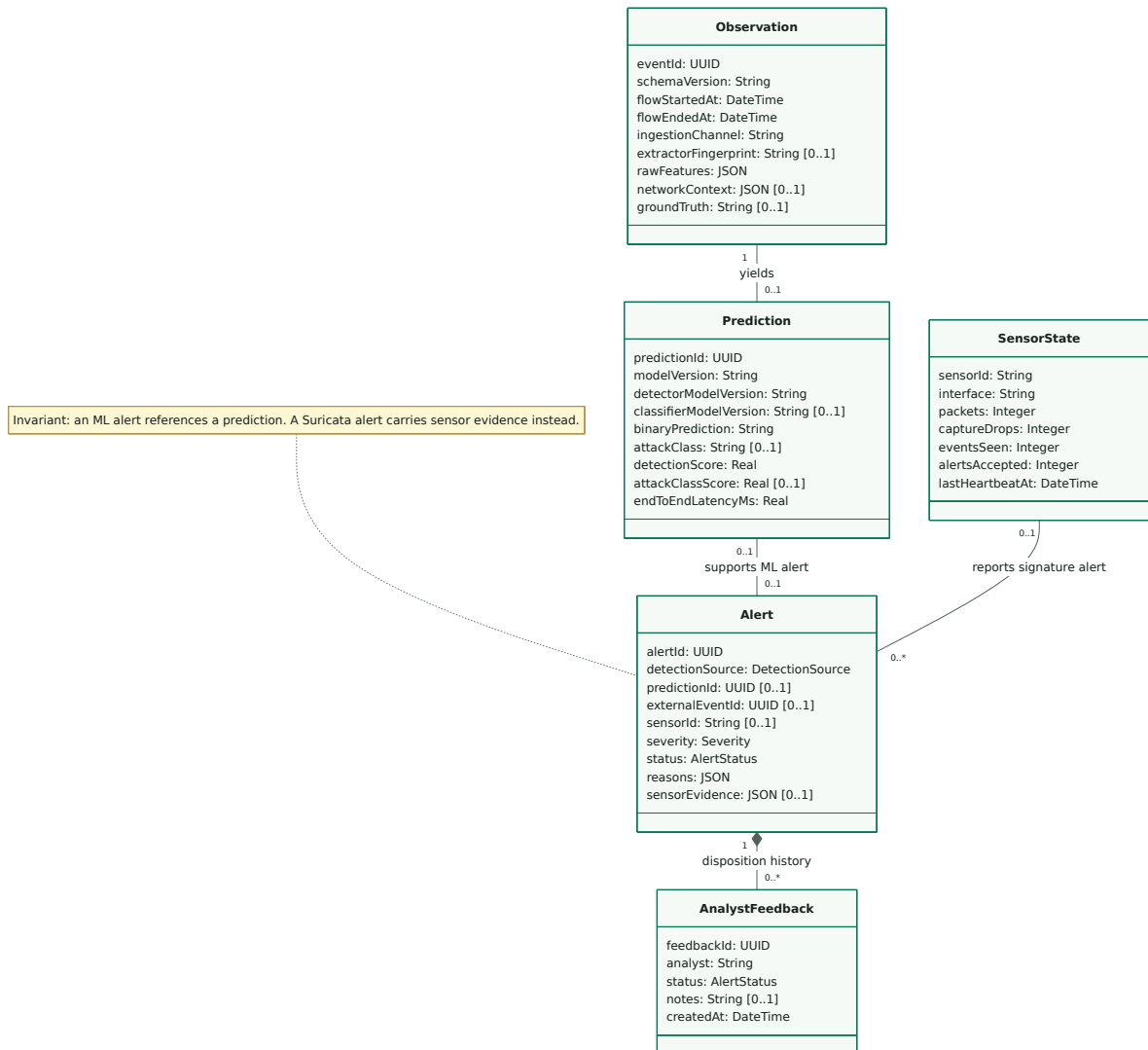


Figure 3.1 – Diagramme de classes du domaine principal

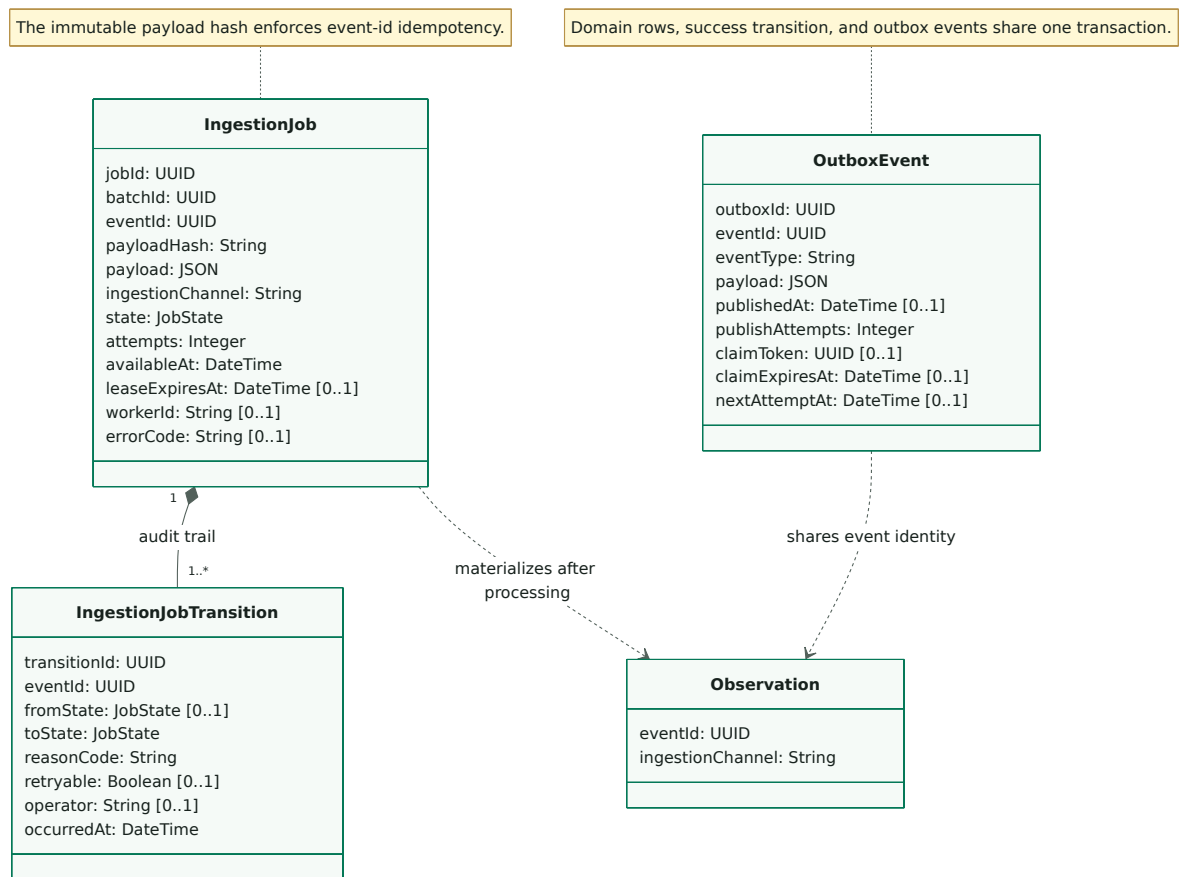


Figure 3.2 – Classes associées à l’ingestion durable

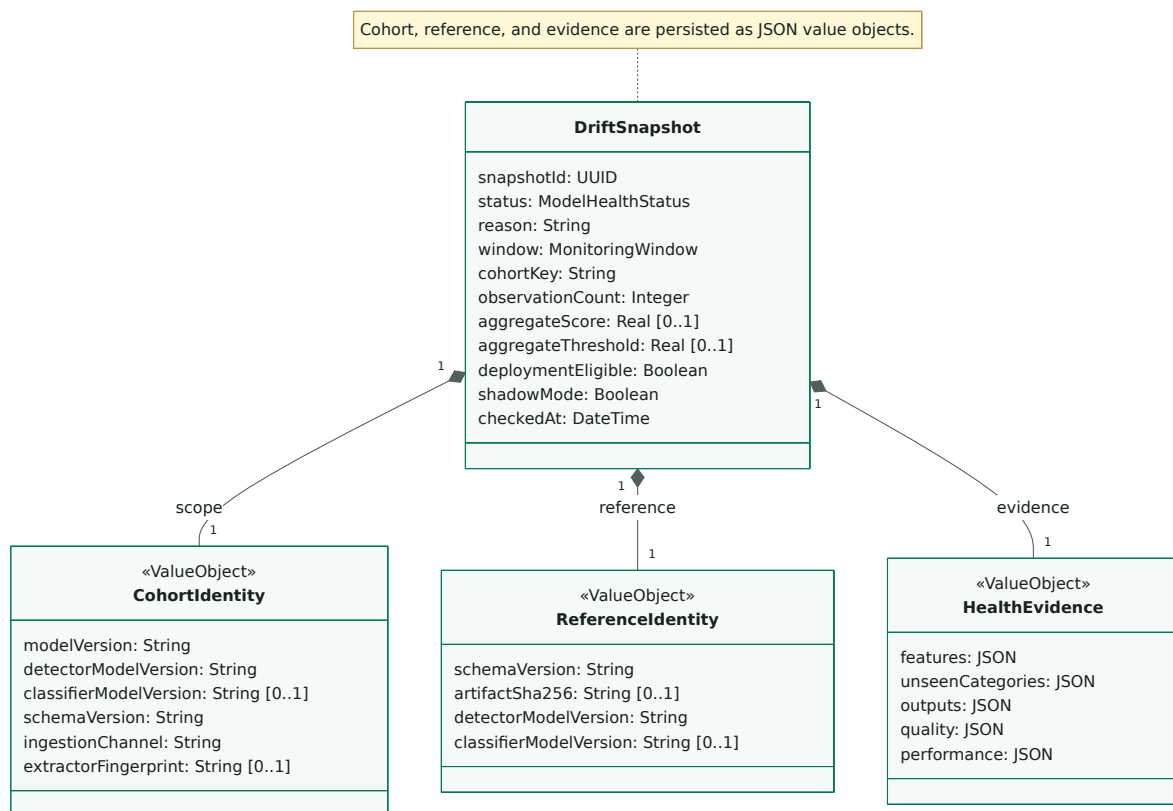


Figure 3.3 – Classes associées au suivi de l'état des modèles

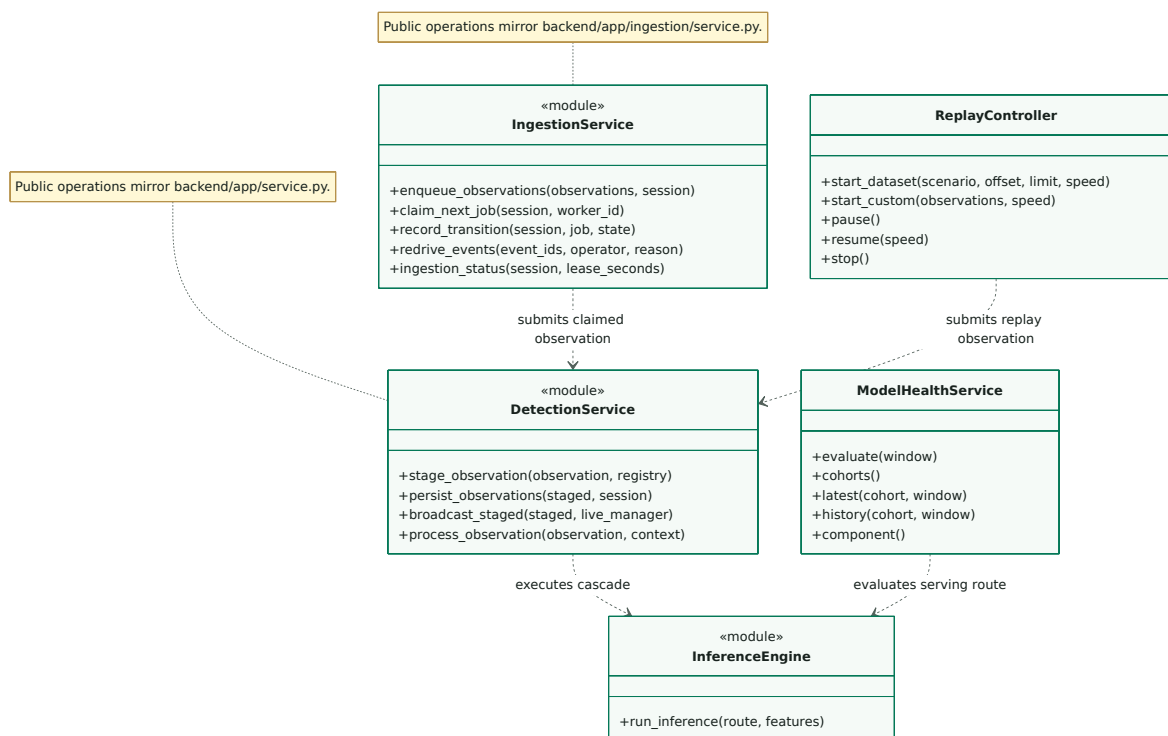


Figure 3.4 – Services et opérations principales

3.2.2 Modèle relationnel

Le modèle relationnel traduit les classes persistantes en tables. Une observation possède une clé primaire `event_id`; une prédiction lui est associée par une clé étrangère unique. Une alerte peut provenir d'une prédiction ML ou d'un capteur Suricata. Les travaux d'ingestion, leurs transitions et les messages d'outbox sont séparés afin de conserver l'idempotence, l'historique et la publication après transaction.

La règle de passage est la suivante : chaque entité devient une table, chaque identifiant devient une clé primaire, les relations un-à-un utilisent une clé étrangère unique et les relations un-à-plusieurs placent la clé étrangère du côté plusieurs. Les objets JSON regroupent les attributs variables sans inventer de tables qui ne sont pas exigées par le comportement.

3.2.3 Dictionnaire de données

Le dictionnaire suivant reprend les champs principaux du modèle. Les champs JSON contiennent les caractéristiques ou les preuves variables; les colonnes de statut et d'identité sont contrôlées par les contrats et contraintes du backend.

Tableau 3.1 – Dictionnaire synthétique des données persistées

Table	Colonne	Type / taille	Contraintes et rôle
-------	---------	---------------	---------------------

observations	event_id	UUID / 36	clé primaire, obligatoire
observations	schema_version	texte / 64	obligatoire, contrat versionné
observations	raw_features	JSON	obligatoire, 83 variables validées
observations	network_context	JSON	facultatif, contexte réseau
predictions	prediction_id	UUID / 36	clé primaire
predictions	event_id	UUID / 36	clé étrangère unique vers observations
predictions	detector_model_version	texte / 128	obligatoire, artefact servi
predictions	binary_prediction	texte / 16	obligatoire, normal ou attack
alerts	alert_id	UUID / 36	clé primaire
alerts	detection_source	texte / 32	obligatoire, ML ou Suricata
alerts	prediction_id	UUID / 36	clé étrangère facultative et unique
ingestion_jobs	event_id	UUID / 36	unique, clé d'idempotence
ingestion_jobs	state	texte / 24	état du cycle de traitement
outbox_events	event_id	UUID / 36	identité de publication

3.3 Architecture de l'application

La vue de composants de la figure 3.5 sépare l'interface, les services applicatifs, la détection et la persistance. Elle ne fait apparaître ni courtier distribué ni service de réputation, car ces composants ne sont pas implémentés.

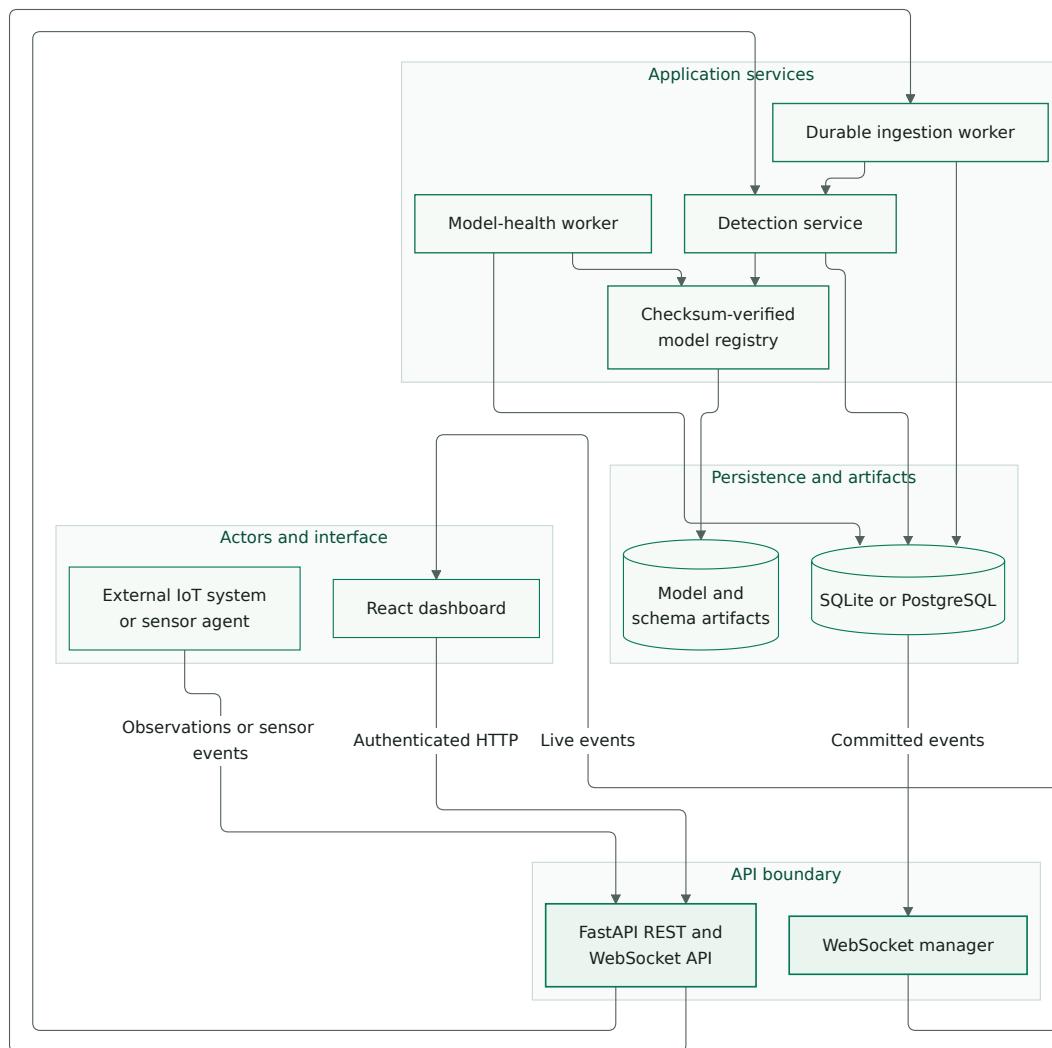


Figure 3.5 – Architecture logicielle par composants

La figure 3.6 représente uniquement le mode de démonstration Docker Compose: une passerelle web, l'API, le travailleur, la migration et PostgreSQL. Le laboratoire Suricata isolé reste un environnement de présentation distinct.

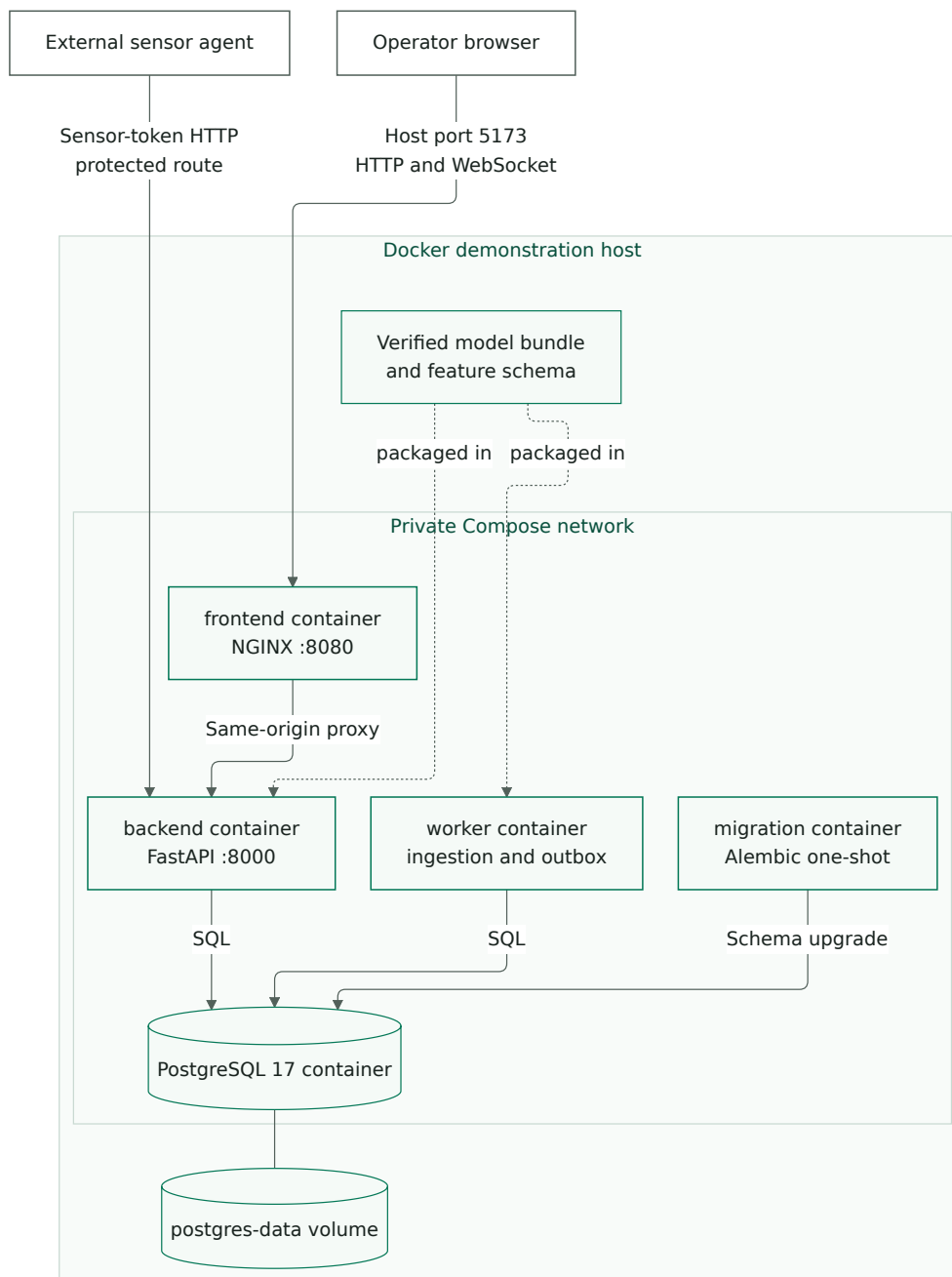


Figure 3.6 – Déploiement du mode de démonstration Docker

3.4 Modélisation dynamique

Les vues dynamiques sont organisées en trois niveaux de lecture. L'activité présente d'abord les objectifs de l'opérateur; les vues d'interaction montrent ensuite les messages échangés; la machine à états isole enfin le cycle de vie persistant d'un travail d'ingestion. Cette séparation évite de répéter les mêmes détails dans chaque diagramme.

3.4.1 Activité de l'opérateur

La figure 3.7 synthétise les choix de l'opérateur sans dupliquer les détails internes des diagrammes de séquence.

3.4.2 Communication pendant l'investigation

La vue de communication de la figure 3.8 privilégie les liens entre objets. La numérotation hiérarchique conserve l'ordre des messages sans reproduire un second diagramme de séquence.

3.4.3 Ingestion durable des observations

La figure 3.9 sépare l'acceptation HTTP, le traitement asynchrone et la diffusion temps réel. Les transactions rendent visibles les frontières d'atomicité, tandis que les fragments alternatifs explicitent le rejet, la réussite et la reprise après erreur. Le cycle de vie persistant correspondant est synthétisé par la figure 3.10.

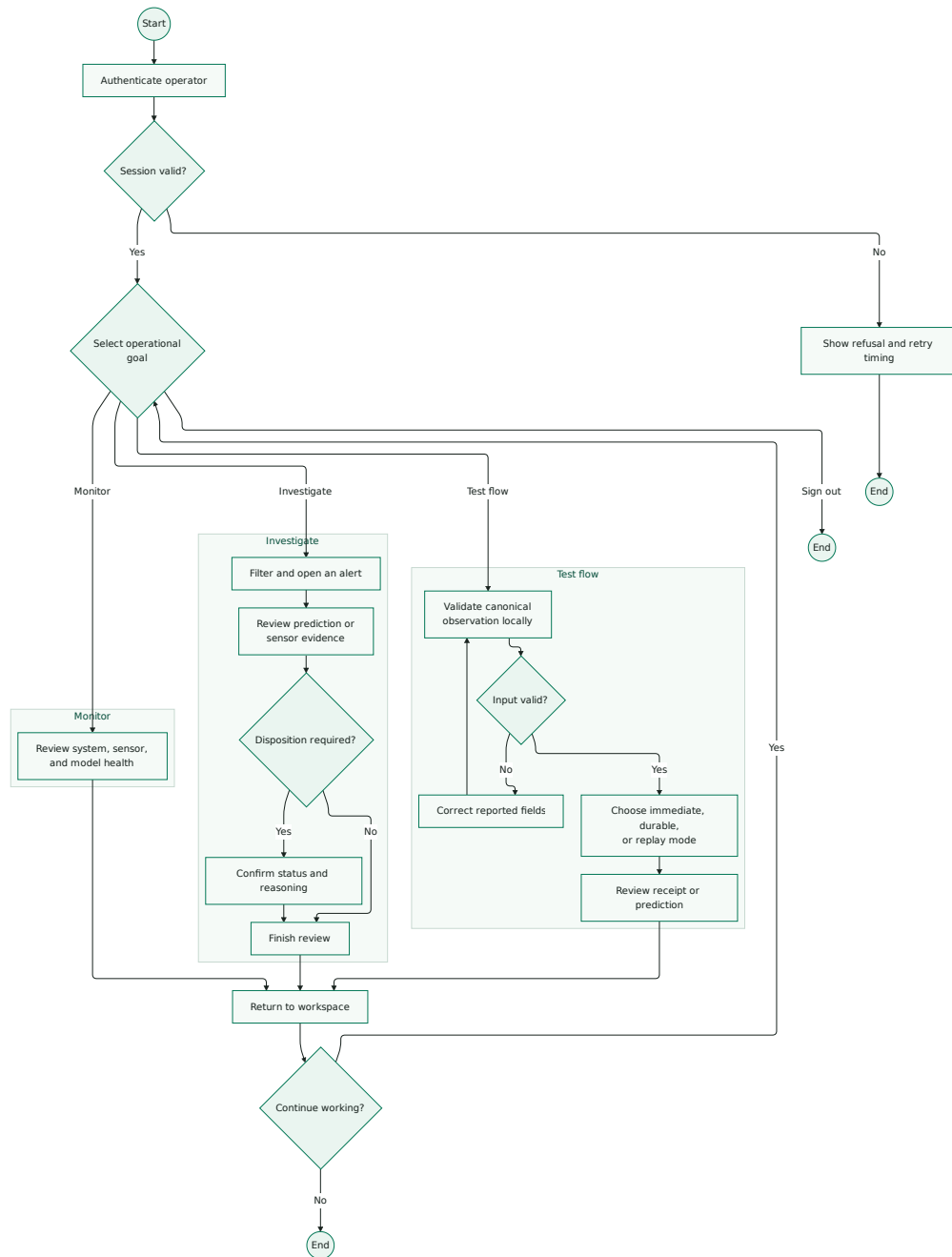


Figure 3.7 – Activité principale de l'opérateur

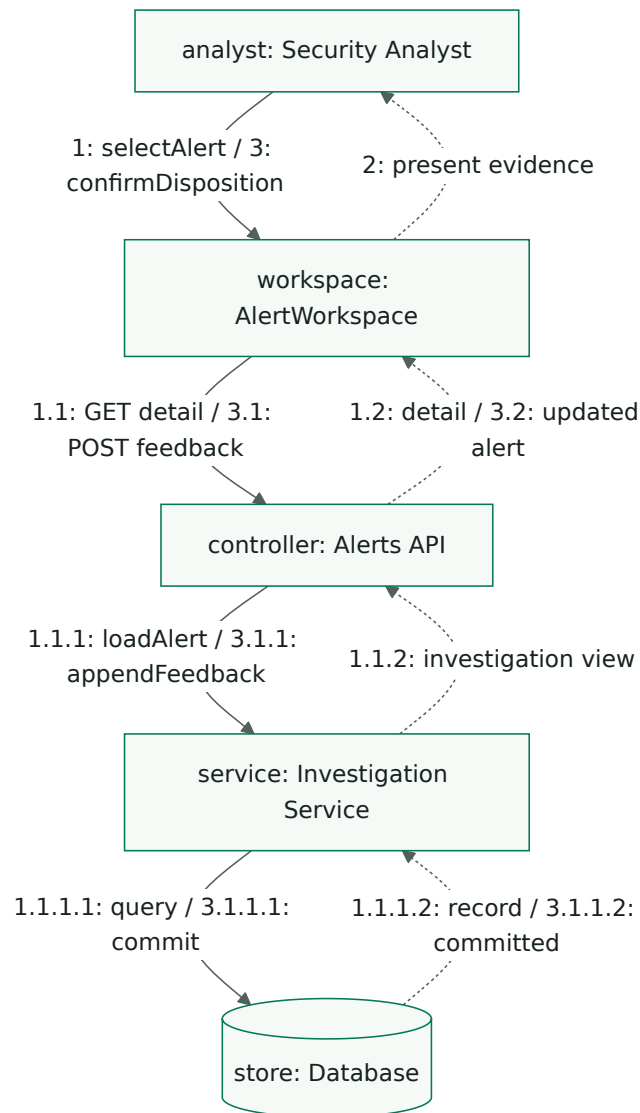


Figure 3.8 – Communication entre objets lors de l’investigation d’une alerte

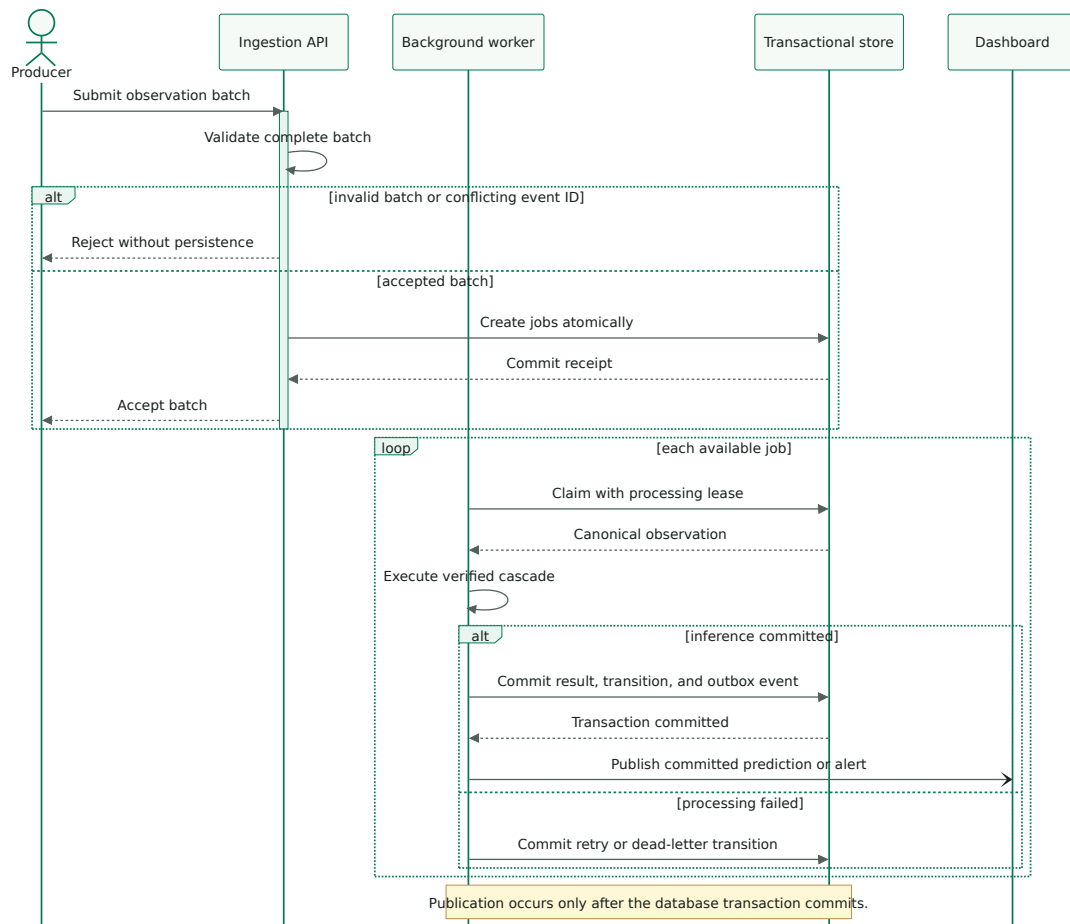


Figure 3.9 – Séquence d’ingestion durable et de diffusion des résultats

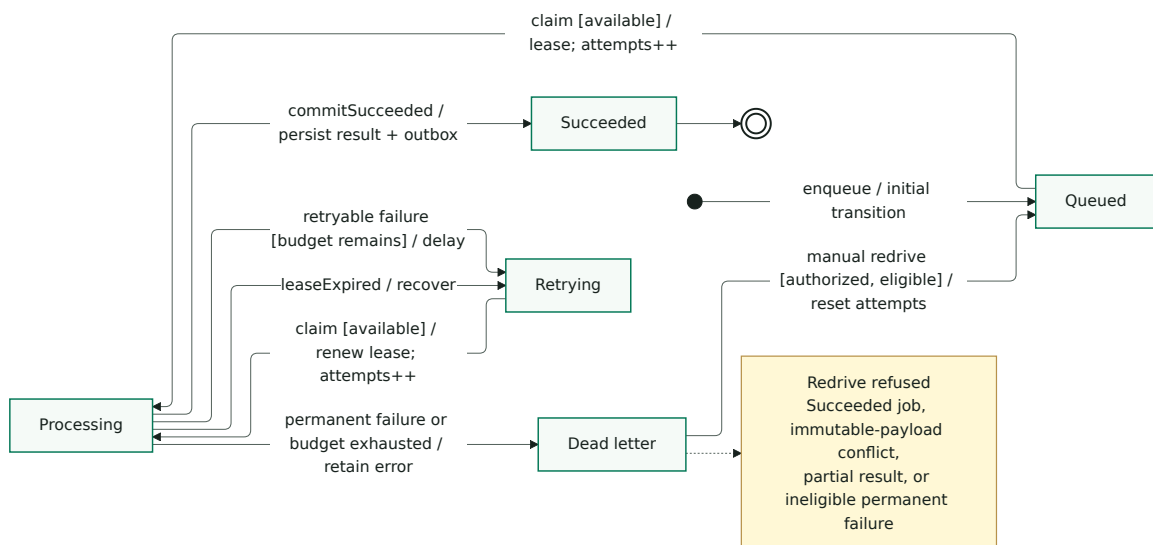


Figure 3.10 – Machine à états d’un travail d’ingestion

3.4.4 Prédiction synchrone

La figure 3.11 isole le parcours central de la recherche: validation du schéma, résolution d'une cascade compatible, inférence puis persistance atomique. Les erreurs techniques internes qui ne modifient pas cette responsabilité ne sont pas reproduites.

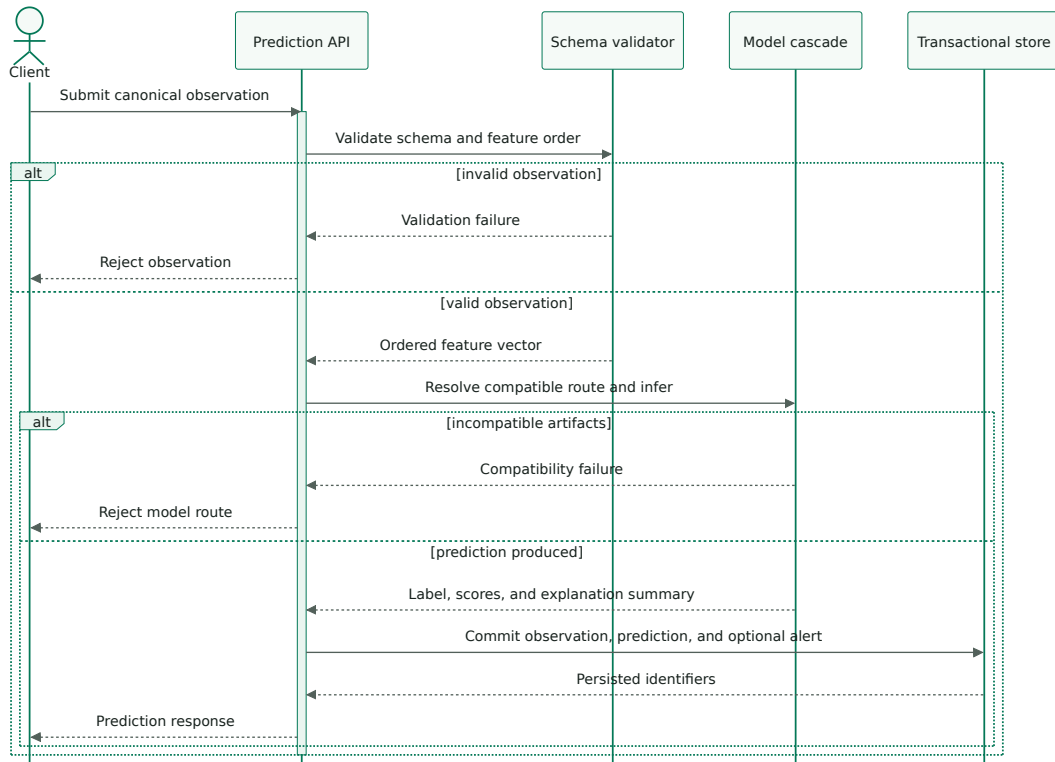


Figure 3.11 – Séquence de prédiction synchrone

3.4.5 Investigation des alertes

L'investigation d'une alerte est détaillée dans la figure 3.12. Elle relie la consultation des preuves à la décision de l'analyste et conserve une trace de l'interprétation humaine du résultat automatique.

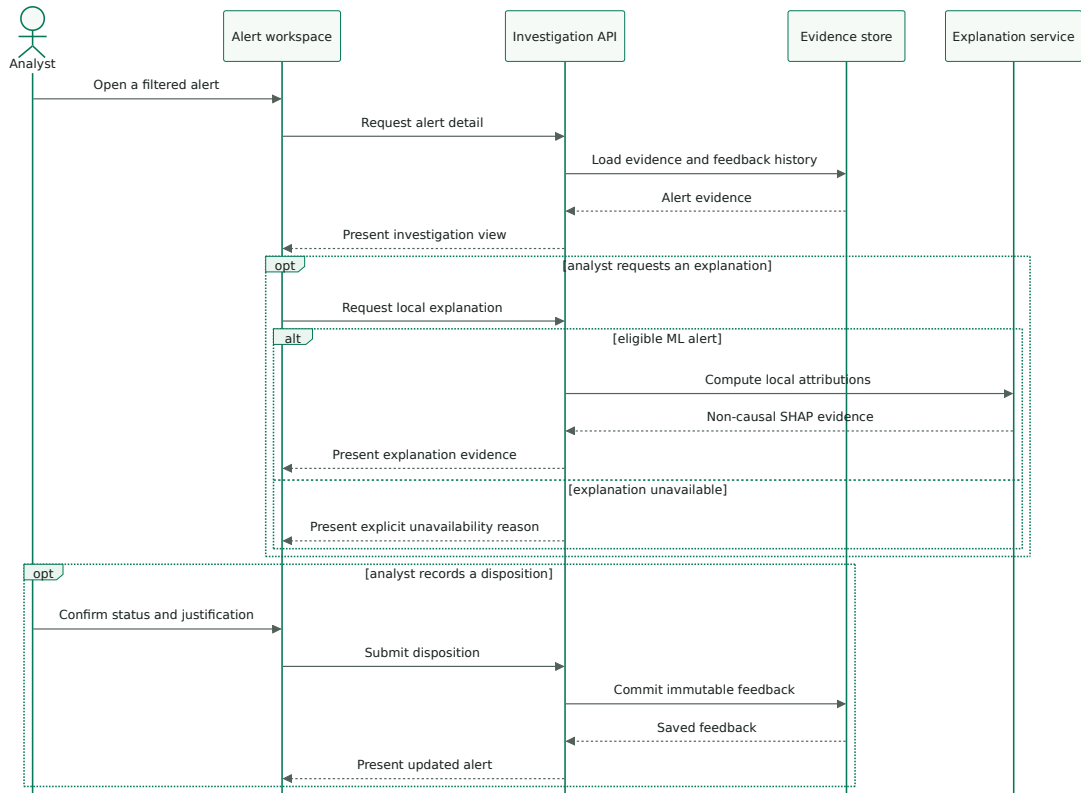


Figure 3.12 – Séquence d'investigation et de qualification d'une alerte

3.4.6 Surveillance de la santé des modèles

Le parcours de la figure 3.13 rassemble les mesures nécessaires à l'examen d'un modèle en service et à la restitution de son état dans l'interface d'administration.

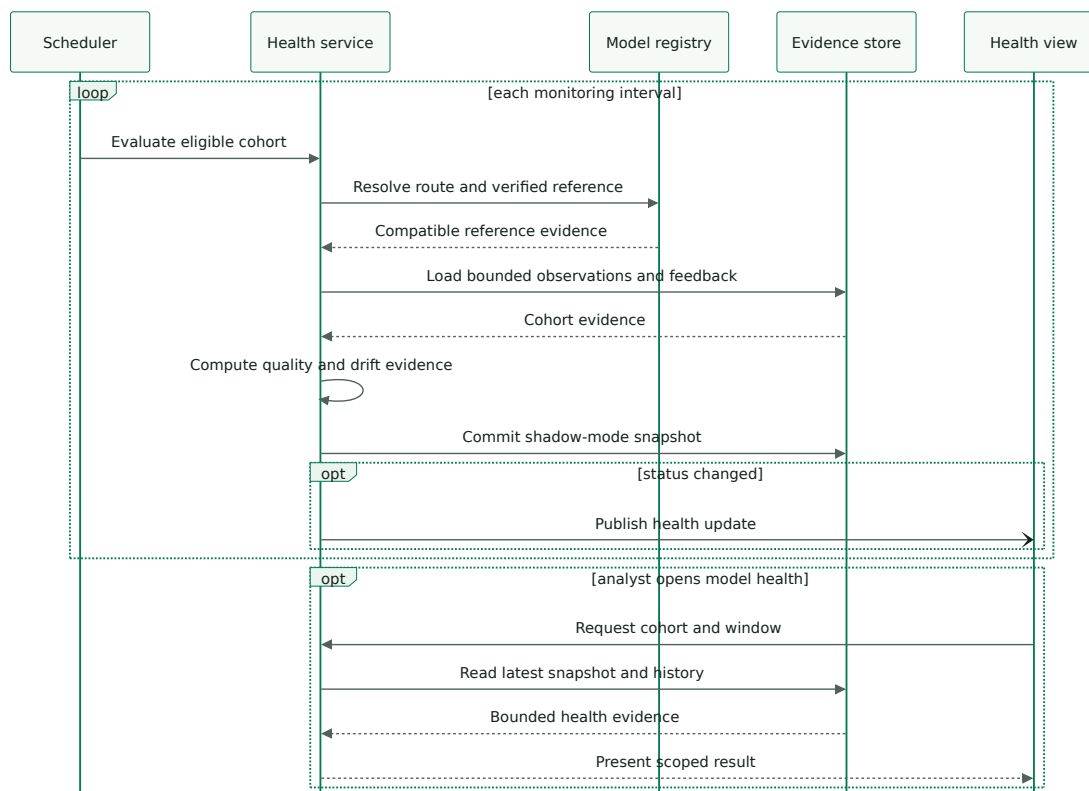


Figure 3.13 – Séquence de consultation de la santé d'un modèle

3.4.7 Scénarios couverts sans diagramme supplémentaire

Trois parcours restent spécifiés sans ajouter de séquence redondante. L'authentification applique un contrôle d'identité avant les fonctions protégées; son comportement est déjà décrit par UC-01. Le rejeu orchestre une suite bornée d'appels au parcours de prédiction de la figure 3.11. Enfin, l'agent Suricata normalise et transmet des alertes de signature, puis réutilise les garanties de transaction et de publication de la figure 3.9. Ces trois cas n'introduisent donc pas une collaboration suffisamment distincte pour justifier un diagramme de séquence supplémentaire.

3.5 Conclusion

La décomposition en vues fonctionnelle, structurelle et dynamique limite la densité de chaque diagramme tout en couvrant les fonctions majeures du système. Les frontières externes, les responsabilités des composants et les scénarios d'erreur sont ainsi visibles sans reproduire les détails d'implémentation dans une seule figure surchargée.

CHAPITRE 4

Réalisation du système

4.1 Introduction

Ce chapitre présente l’environnement reproductible, l’organisation du code et les interfaces essentielles du prototype. Il décrit uniquement les composants présents dans le dépôt et distingue le mode local du mode de démonstration conteneurisé.

4.2 Environnement de développement

4.2.1 Environnement matériel

Le dépôt ne fixe pas un modèle de poste ni une quantité de mémoire comme précondition. Les caractéristiques d’une machine de développement ne sont donc pas inventées ici. La reproductibilité repose sur les fichiers de verrouillage, les images Docker à somme de contrôle et les contrôles préalables qui enregistrent les résultats propres à l’hôte d’exécution.

4.2.2 Environnement logiciel

L’environnement de travail associe Arch Linux, Neovim et les outils du projet. Pacman gère les paquets système, tandis que uv et npm isolent les dépendances applicatives. Le tableau distingue ces outils de production du code des composants nécessaires à l’exécution du prototype.

Tableau 4.1 – Environnement logiciel et rôle des principaux outils

Usage	Outils	Rôle
Poste	Linux, Arch Linux, pacman	Environnement de développement et installation des paquets système.
Édition	Neovim (nvim), terminal, Zsh, Bash	Écriture du code, exécution des commandes et scripts d’automatisation.
Automatisation	Make, Git, GitHub Actions	Commandes communes, historique des modifications et validation continue.
Dépendances	Python 3.12, uv, Node.js, npm	Environnements Python et JavaScript, dépendances et fichiers de verrouillage.

Usage	Outils	Rôle
Calcul et ML	NumPy, pandas, SciPy, scikit-learn, joblib, SHAP	Préparation des données, entraînement, sérialisation et explication des modèles.
Analyse	Jupyter, Matplotlib, nbconvert	Exploration, graphiques scientifiques et export des carnets en HTML.
API	FastAPI, Pydantic, Uvicorn	Validation des entrées et service des routes HTTP et WebSocket.
Données	SQLAlchemy, Alembic, SQLite, PostgreSQL 17	Accès aux données, migrations, transactions et file persistante.
Interface	TypeScript, React 19, Vite	Composants, typage, serveur de développement et compilation de l'interface.
Visualisation	D3, ECharts, Cytoscape, Lucide	Graphiques, topologie d'investigation et icônes.
Tests	pytest, Vitest, Storybook, Playwright, Axe, Ruff	Tests Python, composants, parcours navigateur, accessibilité et analyse statique.
Déploiement	Docker Engine, BuildKit, Docker Compose, NGINX	Construction des images, orchestration locale et passerelle web.
Réseau	Suricata, Mosquitto, NFStream	Signatures sur paquets, trafic MQTT du laboratoire et extraction PCAP hors ligne.
Sécurité et suivi	Argon2id/pwplib, PyJWT, OpenSSL, client Prometheus	Hachage, jetons, génération des secrets et exposition des métriques.
Rapport	LaTeX, latexmk, Mermaid CLI, librsvg	Composition du document et rendu vectoriel des diagrammes.

4.3 Organisation de la réalisation

Machine learning. Le paquet `iot_ids_ml` sépare schéma, préparation, entraînement, évaluation, promotion et validation des artefacts. Le manifeste de production lie modèles, métadonnées et rapport par sommes de contrôle.

Backend. L'application FastAPI sépare routes, validation des caractéristiques, registre de modèles, inférence, ingestion, surveillance et persistance. Le traitement durable associe une boîte de réception, des transitions immuables et une outbox transactionnelle.

Frontend. L'application React est organisée par espaces fonctionnels: supervision, alertes, modèles, laboratoire d'observations et topologie. Les adaptateurs de `api.ts` conservent la séparation entre contrats réseau et composants visuels.

4.4 Principaux choix d'implémentation

4.4.1 Du modèle entraîné à la prédiction servie

La cascade exécute toujours le détecteur binaire. Le classifieur de famille n'est appelé que si le premier étage retourne une attaque; cette dépendance est conservée dans les versions et les mesures retournées. Le registre refuse un artefact dont le schéma, l'ordre des variables ou les sommes de contrôle ne correspondent pas au manifeste actif.

Le même ordre de 83 variables est contrôlé à l'entraînement et à l'inférence. Le rejeu permet d'exercer cette chaîne avec les données enregistrées sans réentraîner le modèle. Les explications SHAP sont calculées à la demande pour l'investigation ; elles décrivent les contributions au résultat du modèle.

4.4.2 Ingestion durable et traitement des erreurs

L'ingestion HTTP valide un lot complet avant toute insertion. L'identifiant d'événement et l'empreinte du contenu assurent l'idempotence. Le travailleur réserve ensuite un travail, exécute la cascade et valide dans une transaction l'observation, la prédiction, l'alerte éventuelle, la transition et les messages d'outbox. La diffusion temps réel ne commence qu'après cette validation.

Une nouvelle soumission identique n'ajoute pas une seconde observation ; un contenu différent sous le même identifiant est refusé. Après une erreur temporaire, le travail est repris avec un délai et un nombre d'essais bornés. Les échecs persistants sont isolés pour permettre leur examen et une relance manuelle authentifiée, avec conservation du motif.

4.4.3 Interface, contrôle d'accès et provenance

L'interface conserve trois portées distinctes: les agrégats persistés du tableau de bord, la page d'alertes chargée et les événements reçus pendant la session. Cette distinction évite de présenter une activité locale du navigateur comme une mesure historique globale.

La session opérateur utilise un jeton à durée limitée ; l'interface traite son expiration et les refus de l'API. L'identité associée à une qualification provient du jeton validé, et non d'un nom libre fourni par le navigateur. Les vues présentent les états de chargement, d'erreur et de reconnexion pour éviter qu'une donnée ancienne soit confondue avec un état actualisé.

Dans le laboratoire, Suricata observe le trafic MQTT et HTTP du réseau Docker isolé. L'agent transmet les événements avec une identité machine distincte. L'interface affiche leur origine de signature ; ils ne traversent pas la cascade RT-IoT2022. NFStream reste une voie d'extraction PCAP hors ligne distincte.

4.4.4 Exécution reproductible

`make setup` prépare les dépendances ; `make verify-model` contrôle le bundle promu. Le mode `make demo` prépare un rejeu initial dans une base temporaire. Le laboratoire complet démarre avec `make demo-sensor`. Ces modes alternatifs facilitent une démonstration maîtrisée. Les migrations sont appliquées avant le démarrage des services, et les sondes de santé permettent de vérifier leur disponibilité.

4.5 Principales interfaces graphiques

4.5.1 Supervision opérationnelle

La figure 4.1 regroupe l'état de la chaîne de service, la progression du rejeu et les indicateurs d'ingestion. Les valeurs sont associées à leur portée afin de distinguer historique persisté et activité de la session.

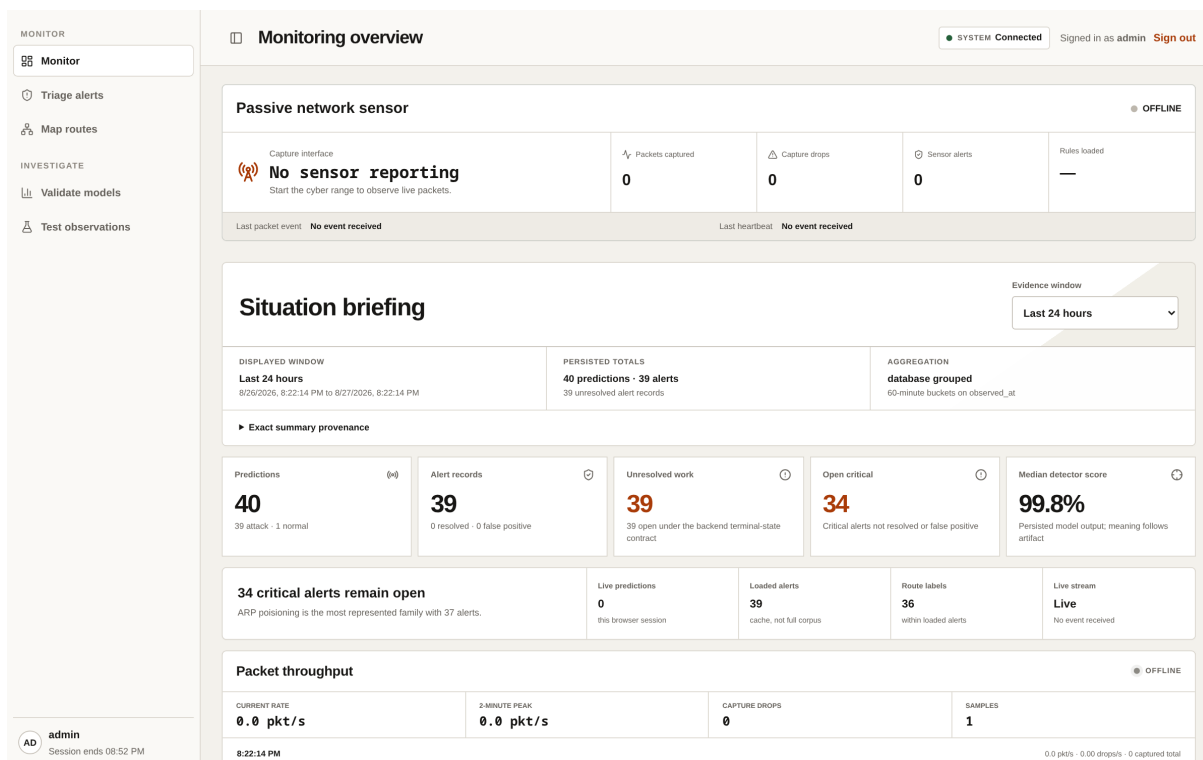


Figure 4.1 – Espace principal de supervision du prototype

4.5.2 Investigation des alertes

La figure 4.2 présente la file filtrable et le point d'entrée de l'investigation. Le détail distingue l'origine ML de l'origine Suricata et conserve la provenance, les scores, les versions et l'historique de qualification disponibles.

Alert investigation SYSTEM Connected Signed in as admin [Sign out](#)

Security alerts 39
matching alerts · newest first

Quick views: **All alerts** Needs review Critical In progress

Search: Time: Detection: Severity: Status:

SEVERITY	DETECTION	ROUTE	PROTOCOL	EVIDENCE	STATUS	TIME
CRITICAL	ARP_poisoning 7beb756b-8792-4df1-a66d-8bee443ce55e	port 54835 to port 53	udp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 25b99088-e833-4773-9b24-f79225459540	port 38680 to port 53	udp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning b88c40ba-2715-4191-846d-c39a496069ad	port 46956 to port 443	tcp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 65006096-278c-4f28-9b83-89619ed15e19	port 46962 to port 443	tcp	99.9% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 13fd6d85-dd61-4cab-bf48-e27a623ec25b	port 46958 to port 443	tcp	99.9% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 58131d04-3453-4c40-8d68-9fcfc3d145cd	port 46960 to port 443	tcp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning b8da7703-24f0-45f9-0022-a2d5a9254b89	port 5353 to port 5353	udp	98.6% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 0d552830-4bf4-467e-89b3-264df82ad385	port 5353 to port 5353	udp	98.6% model	New	07:22:13 PM

AD admin
Session ends 08:52 PM

Figure 4.2 – Espace d'investigation des alertes

4.5.3 Analyse des modèles

La figure 4.3 expose les artefacts actifs et leur protocole d'évaluation. Les mesures sont présentées avec leur partage, leurs graines et leurs limites pour ne pas transformer un score interne au jeu de données en promesse de déploiement.

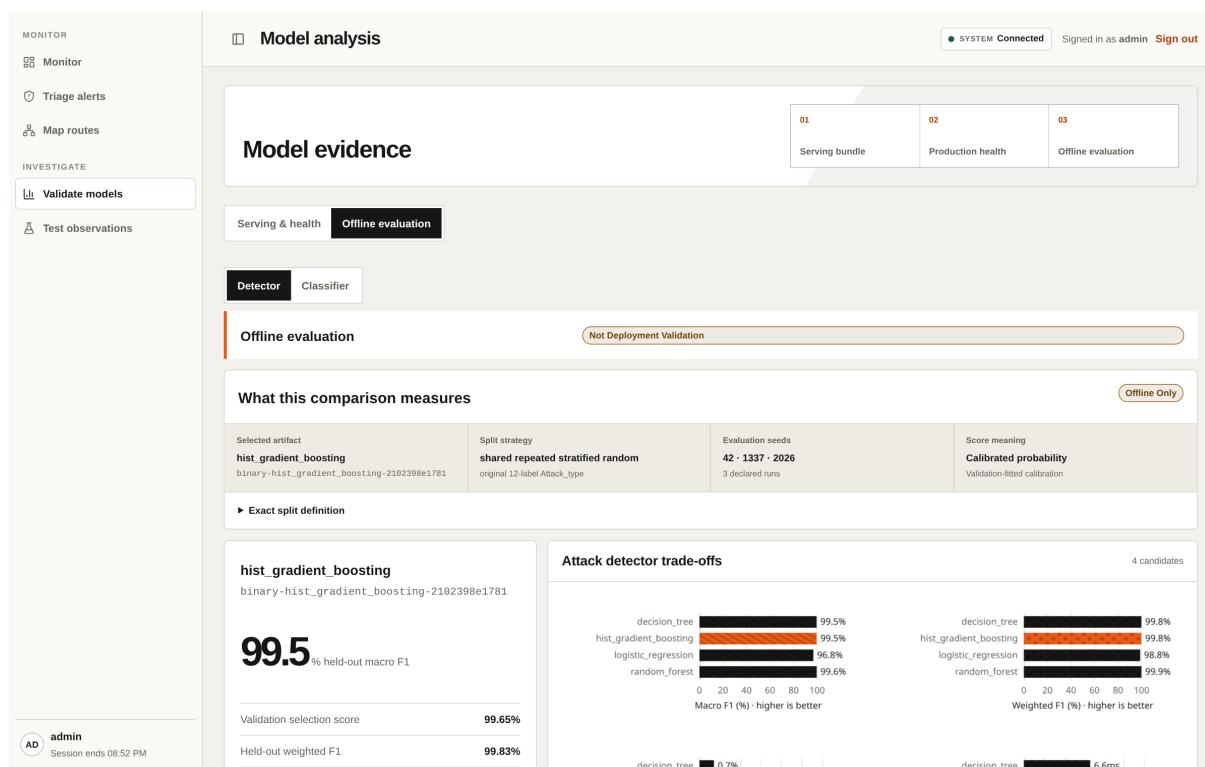


Figure 4.3 – Espace d'analyse et de traçabilité des modèles

4.5.4 Laboratoire d'observations

La figure 4.4 permet de contrôler une observation ou un petit lot avant soumission. La validation locale fournit un retour précis, mais l'API authentifiée reste l'autorité finale pour l'acceptation.

MONITOR

- Monitor
- Triage alerts
- Map routes

INVESTIGATE

- Validate models
- Test observations**

Observation lab

SYSTEM Connected Signed in as admin Sign out

Observation lab

1 Choose observations

RT-IoT2022 CSV file
CSV only - maximum 10 MB - one file

Choose File No file chosen

Or drop one CSV here

Load verified normal example Load verified attack example

RT-IoT2022 extracted SHA-256
956956c09c1764584fa08acd9f6876475626bcdcd6a6b1f8c492c2e9a2b89ea. Source lines 2 and 12509; only the dataset index outside the serving schema is removed.

SELECTED FILE
rt-iot2022-line-12509-attack.csv

ROWS	FEATURE COLUMNS	REFERENCE LABELS
1	83	1

2 Review contract and values

Header contract 63 feature names in canonical order

Cell values 81 numeric and 2 categorical fields validated per row

Sample preview First 1 rows - selected identifying and flow fields

Row	id.orig_p	id.resp_p	proto	service	flow_duration	fwd_pkts_tot	bwrd_pkts_tot	Attack_type
1	5353	5353	udp	dns	0	1	0	ARP_poisk

The complete 83-feature vector is submitted; this focused preview supports recognition without presenting an 84-column page.

3 Choose processing path

☒ **Analyze immediately**
Persists predictions, creates alerts for attack verdicts, and returns every result in this workspace.
Best for: Controlled validation and row-by-row comparison
LIMIT 10,000 ROWS

☐ **Queue reliably**
Persists jobs for worker processing, retry, dead-letter recovery, and outbox delivery evidence.
Best for: Recoverable asynchronous ingestion
LIMIT 1,000 ROWS

☐ **Replay as live traffic**
Starts a custom replay through the detector, persistence layer, and live event stream.
Best for: End-to-end exercises using uploaded observations
LIMIT 100,000 ROWS

admin Session ends 08:52 PM

Figure 4.4 – Laboratoire de validation et de soumission d'observations

4.6 Validation technique

La validation automatisée couvre plusieurs niveaux. Les tests Python contrôlent préparation, schémas, promotion, registre, API, persistance, ingestion, rejeu, santé des modèles et Suricata. Les tests Vitest couvrent les adaptateurs et composants React. Les scénarios Playwright exercent le parcours de bout en bout avec les services et les artefacts réels. Les commandes `make test`, `make lint`, `make build` et `make project-preflight` constituent les portes de validation déclarées par le dépôt.

Storybook complète cette validation par des interactions sur les composants isolés. Des contrôles comparent également les chiffres du support de maintenance au rapport d'évaluation promu. La répétition complète du 7 septembre 2026 a validé les dix parcours navigateur, le déploiement Docker temporaire et les trois scénarios de signature Suricata. Le carnet exécuté et son export HTML conservent les résultats de cette répétition ; ils attestent du fonctionnement mesuré, sans démontrer une efficacité sur un réseau externe.

4.7 Conclusion

La réalisation conserve une séparation nette entre apprentissage, service, persistance et interface. Les mécanismes de validation et de traçabilité rendent la démonstration reproductible, sans supprimer les limites scientifiques et opérationnelles rappelées dans la conclusion générale.

CHAPITRE 5

Résultats expérimentaux

5.1 Objectif et source de preuve

Ce chapitre restitue le rapport d'évaluation promu avec les modèles servis. Toutes les valeurs proviennent du fichier `models/production/evaluation-report.json`, lié aux artefacts par sommes SHA-256. Le fichier source RT-IoT2022 contient 123 117 lignes et 83 variables de modèle. Son empreinte commence par 956956c09c17; elle correspond au manifeste de données du projet.

5.2 Qualité, nettoyage et déséquilibre

L'audit ne trouve aucune valeur manquante ou infinie. Il identifie toutefois six signatures de variables associées à des labels contradictoires, soit 128 lignes. Toutes ces lignes sont supprimées afin de ne pas choisir arbitrairement un label. Le nettoyage conserve ensuite un seul exemplaire de 5 080 doublons de même label.

La distribution source est très déséquilibrée: `DOS_SYN_Hping` représente 94 659 lignes, alors que `NMAP_FIN_SCAN` n'en contient que 28 et `Metasploit_Brute_Force_SSH` 37. Pour cette raison, l'exactitude globale n'est pas utilisée seule; la macro-F1, le rappel par classe, le support et le taux de faux positifs sont publiés.

L'analyse descriptive montre en outre des variables très asymétriques et de nombreux zéros. Les graphiques emploient donc une transformation $\log_{1p}$ uniquement pour la visualisation. Les associations avec la cible restent descriptives: elles peuvent révéler des artefacts de scénario et ne sont pas interprétées comme des causes d'attaque.

5.3 Protocole expérimental

Après nettoyage, chaque graine (42, 1337 et 2026) produit un partage stratifié 60/20/20 en entraînement, validation et test. Les indices sont communs aux candidats d'une graine. Le prétraitement et la calibration sigmoïde par validation croisée sont ajustés uniquement sur l'entraînement. La sélection maximise la macro-F1 moyenne de validation; le taux de faux positifs, la taille sérialisée et le nom stable départagent ensuite les candidats. Le test ne participe jamais à ce choix.

Le fichier ne fournit aucun identifiant de session, d'appareil ou horodatage dont la sémantique soit assez fiable pour construire un partage temporel ou groupé. Le protocole est donc une référence interne répétée, non une validation de déploiement.

5.4 Comparaison des classifieurs

Tableau 5.1 – Comparaison du détecteur binaire sur les trois graines

Modèle	F1 validation	σ	FPR validation	F1 test 42
Régression logistique	0,9702	0,0028	5,5371%	0,9684
Arbre de décision	0,9947	0,0006	0,7494%	0,9945
Forêt aléatoire	0,9954	0,0006	0,9992%	0,9949
Gradient boosting	0,9955	0,0003	0,8604%	0,9961

Tableau 5.2 – Comparaison du classifieur des familles d'attaque

Modèle	F1 validation	σ	FPR macro	F1 test 42
Régression logistique	0,6222	0,0745	0,2357%	0,6308
Arbre de décision	0,9375	0,0060	0,0124%	0,9602
Gradient boosting	0,9659	0,0070	0,0072%	0,9619
Forêt aléatoire	0,9668	0,0039	0,0104%	0,9642

Le gradient boosting est retenu pour la détection et la forêt aléatoire pour les familles. Les écarts de validation entre les meilleurs candidats sont faibles; la règle de sélection déclarée, et non le score de test, détermine les champions. La figure 5.1 montre la restitution graphique générée par le notebook exécutable.

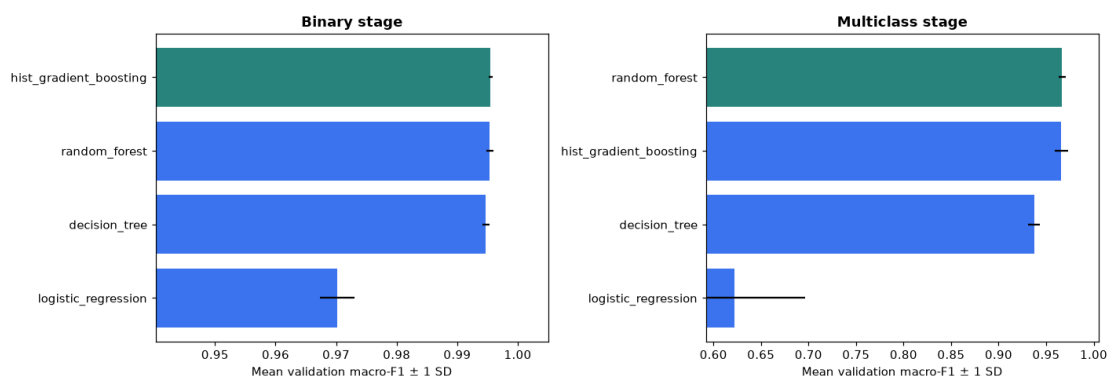


Figure 5.1 – Comparaison des candidats dans le notebook d'analyse

5.5 Expérience sur le déséquilibre

Une expérience diagnostique conserve l'architecture HistGradientBoosting et les mêmes partitions de la graine 42. Elle compare l'absence de correction, la pondération des classes et un sous-échantillonnage aléatoire limité à l'entraînement. Cette expérience ne participe pas à la promotion.

Tableau 5.3 – Comparaison contrôlée des traitements du déséquilibre

Étage	Traitement	Lignes fit	F1 validation	F1 test
Binaire	Sans correction	70 745	0,9955	0,9969
Binaire	Pondération	70 745	0,9946	0,9961
Binaire	Sous-échantillonnage	14 412	0,9891	0,9918
Familles	Sans correction	63 539	0,9528	0,9668
Familles	Pondération	63 539	0,9368	0,9551
Familles	Sous-échantillonnage	153	0,7150	0,7150

La pondération améliore légèrement le rappel de la classe binaire minoritaire (0,9975 contre 0,9946 sans correction), sans maximiser la macro-F1. Le sous-échantillonnage conserve le support de la classe la plus rare pour chaque classe et détruit donc trop d'information, surtout au second étage où seules 153 lignes subsistent. Cette comparaison montre qu'un résultat agrégé doit toujours être lu avec le rappel et le support des classes rares.

5.6 Erreurs des champions et de la cascade

La matrice binaire du test 42 contient 21 159 attaques correctement détectées, 20 attaques manquées, 14 faux positifs et 2 389 flux normaux correctement reconnus. Le taux de faux positifs vaut ainsi 0,5826%.

Tableau 5.4 – Classes les plus fragiles dans la cascade, graine 42

Classe	Support	Rappel	F1
NMAP_FIN_SCAN	6	0,8333	0,7692
Metasploit_Brute_Force_SSH	7	0,7143	0,8333
DDOS_Slowloris	106	0,9528	0,9573
NMAP_TCP_scan	200	0,9950	0,9975

La cascade complète atteint 0,9520 de macro-F1 pour la graine 42 et 0,9686 en moyenne sur les trois graines. Le score 42 est sensible à une seule erreur parmi six ou sept observations rares. Les supports sont donc indissociables des scores; la valeur 0,9520 ne traduit pas une baisse uniforme sur les classes fréquentes.

5.7 Calibration, seuil et coût opérationnel

Les deux champions sont calibrés par sigmoid sur l’entraînement. Sur le test 42, le détecteur obtient un Brier de 0,0020 et une erreur de calibration attendue de 0,0008; le classifieur obtient respectivement 0,0015 et 0,0010. Ces nombres ne garantissent pas une calibration sur un autre réseau. Le seuil de service 0,5 est documenté avec une courbe validation reliant faux positifs et attaques manquées.

Sur la machine de mesure, le p95 moyen par observation vaut 19,27 ms pour le détecteur et 31,79 ms pour le classifieur. Les artefacts occupent 418 481 et 1 325 252 octets. Ces mesures comparent les candidats sur un hôte donné et ne constituent pas un objectif de capacité en production.

5.8 Menaces à la validité

Les résultats restent internes à RT-IoT2022. Les classes rares rendent certaines mesures instables; les doublons et artefacts de scénario peuvent faciliter la séparation; aucune validation temporelle, par appareil, inter-réseau ou inter-jeu n’est disponible. La compatibilité de noms avec un extracteur NFStream ne prouve pas l’équivalence des valeurs. Enfin, la calibration et la latence sont dépendantes du contexte de mesure. Ces limites interdisent de présenter le prototype comme un contrôle de sécurité validé sur un réseau réel.

5.9 Conclusion

L’expérience répond au cahier des charges: elle analyse les données, compare quatre modèles, traite et mesure le déséquilibre, puis évalue les deux étages et la cascade. Le meilleur résultat reste une référence reproductible interne; une collecte autorisée, groupée par session et séparée chronologiquement est la prochaine preuve scientifique nécessaire.

Conclusion générale

Ce projet d'été avait pour objectif de construire une chaîne reproductible de détection d'intrusions à partir d'observations de flux IoT. Le travail a abouti à un prototype cohérent qui vérifie la provenance du jeu de données, entraîne et promeut une cascade binaire puis multiclasse, valide les contrats d'entrée, conserve observations, prédictions et alertes, et fournit une interface d'investigation. L'ingestion durable ajoute l'idempotence, les reprises contrôlées et une diffusion post-transactionnelle; le laboratoire Suricata constitue une voie de signature distincte des prédictions du modèle.

Sur le partage de test RT-IoT2022 déclaré, la cascade complète atteint un macro-F1 de 0,9520 pour la graine 42 et une moyenne de 0,9686 sur les trois graines évaluées. Ces valeurs forment une référence reproductible interne au jeu de données, non une preuve d'efficacité sur un réseau réel. Les limites principales restent l'absence de corpus NFStream natif étiqueté, l'absence d'évaluation temporelle ou externe et l'absence de mesure de capacité sur un matériel cible. La suite logique consiste donc à collecter des flux autorisés et représentatifs, figer des partitions par session, évaluer la chaîne sur ces données puis décider, à partir de preuves mesurées, si une expérimentation de déploiement peut être envisagée.

Bibliographie et webographie

- Zied Aouini and Adrian Pekar. NFStream: A flexible network data analysis framework. *Computer Networks*, 204:108719, February 2022. ISSN 13891286. doi: 10.1016/j.comnet.2021.108719. URL <https://linkinghub.elsevier.com/retrieve/pii/S1389128621005739>.
- Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, October 2001. ISSN 0885-6125, 1573-0565. doi: 10.1023/A:1010933404324. URL <https://link.springer.com/10.1023/A:1010933404324>.
- Marco Cantone, Claudio Marrocco, and Alessandro Bria. Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study. *IEEE Access*, 12:144489–144508, 2024. ISSN 2169-3536. doi: 10.1109/ACCESS.2024.3472907. URL <https://ieeexplore.ieee.org/document/10704637/>.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, October 2001. ISSN 0090-5364. doi: 10.1214/aos/1013203451. URL <https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full>.
- Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, December 2012. ISSN 1556-4681, 1556-472X. doi: 10.1145/2382577.2382579. URL <https://dl.acm.org/doi/10.1145/2382577.2382579>.
- Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning - ICML '05*, pages 625–632, Bonn, Germany, 2005. ACM Press. ISBN 978-1-59593-180-1. doi: 10.1145/1102351.1102430. URL <https://dl.acm.org/doi/10.1145/1102351.1102430>.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Karen Scarfone and Peter Mell. Guide to Intrusion Detection and Prevention Systems (IDPS). Technical Report NIST SP 800-94, National Institute of Standards and Technol-

- ogy, Gaithersburg, MD, 2007. URL <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf>.
- B. S. Sharmila and Rohini Nagapadma. RT-IoT2022, 2023a. URL <https://archive.ics.uci.edu/dataset/942>.
- B S Sharmila and Rohini Nagapadma. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity*, 6(1):41, September 2023b. ISSN 2523-3246. doi: 10.1186/s42400-023-00178-5. URL <https://cybersecurity.springeropen.com/articles/10.1186/s42400-023-00178-5>.
- Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, July 2009. ISSN 03064573. doi: 10.1016/j.ipm.2009.03.002. URL <https://linkinghub.elsevier.com/retrieve/pii/S0306457309000259>.
- Paul Watrobski, Murugiah Souppaya, Joshua Klosterman, and William Barker. Methodology for characterizing network behavior of internet of things devices. Technical Report NIST IR 8349, National Institute of Standards and Technology (U.S.), Gaithersburg, MD, August 2025. URL <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8349.pdf>.
- Bruno Bogaz Zarpelão, Rodrigo Sanches Miani, Cláudio Toshio Kawakani, and Sean Carliso De Alvarenga. A survey of intrusion detection in Internet of Things. *Journal of Network and Computer Applications*, 84:25–37, April 2017. ISSN 10848045. doi: 10.1016/j.jnca.2017.02.009. URL <https://linkinghub.elsevier.com/retrieve/pii/S1084804517300802>.